Linux Systems and Performance

PART 2: CPU Performance and Scheduling

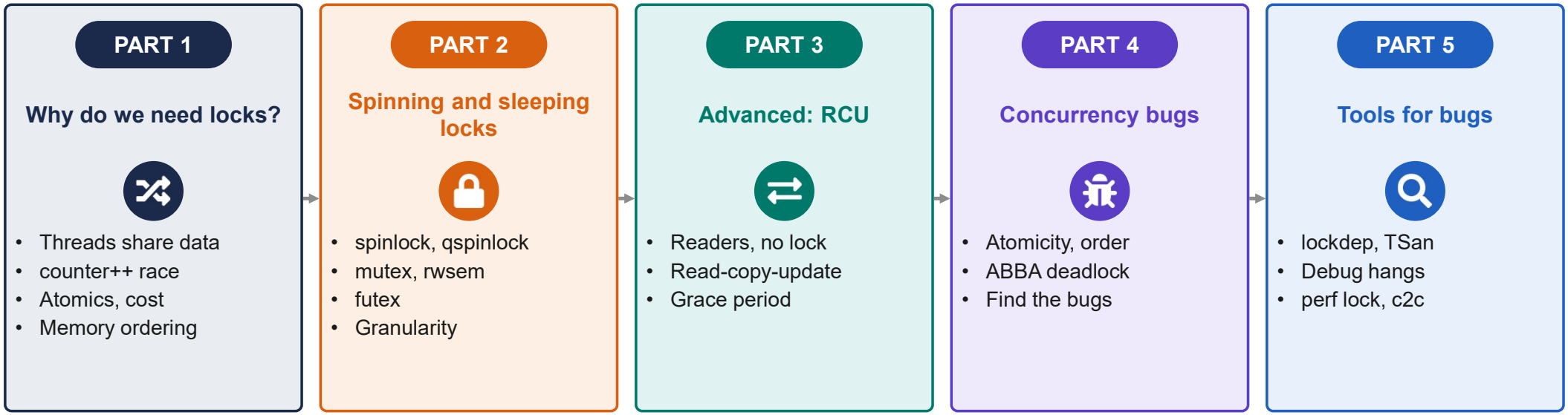
Week 6: Concurrency in Linux — Locks, Synchronization and Bugs

Youngjae Kim, Ph.D.

Data-Intensive AI Computing and Systems (DISCOS) Laboratory
Department of Computer Science and Engineering, Sogang University
URL: https://discos.sogang.ac.kr/
E-mail: youkim@sogang.ac.kr



Roadmap of This Week



This week

Why threads that share data need locks, the spinning and sleeping locks Linux gives us (spinlock, mutex, futex, condition variables), an advanced lock for read-mostly data (RCU), the bugs locks cause, and the tools that find these bugs.

How Does This Relate to webOS?



Race

Lost update, corrupted list — a crash that appears once a week



Deadlock

Two services wait for each other forever — LS2 calls never return



Lock contention → jank

The UI thread waits for a lock held during slow disk I/O



Priority inversion over IPC

A high-priority client waits for a low-priority service

1. **Why locks** — race, atomics, ordering
2. **Locks** — spin & sleep, futex
3. **RCU** — advanced: no reader lock
4. **Bugs** — race, order, deadlock
5. **Tools** — find the bugs



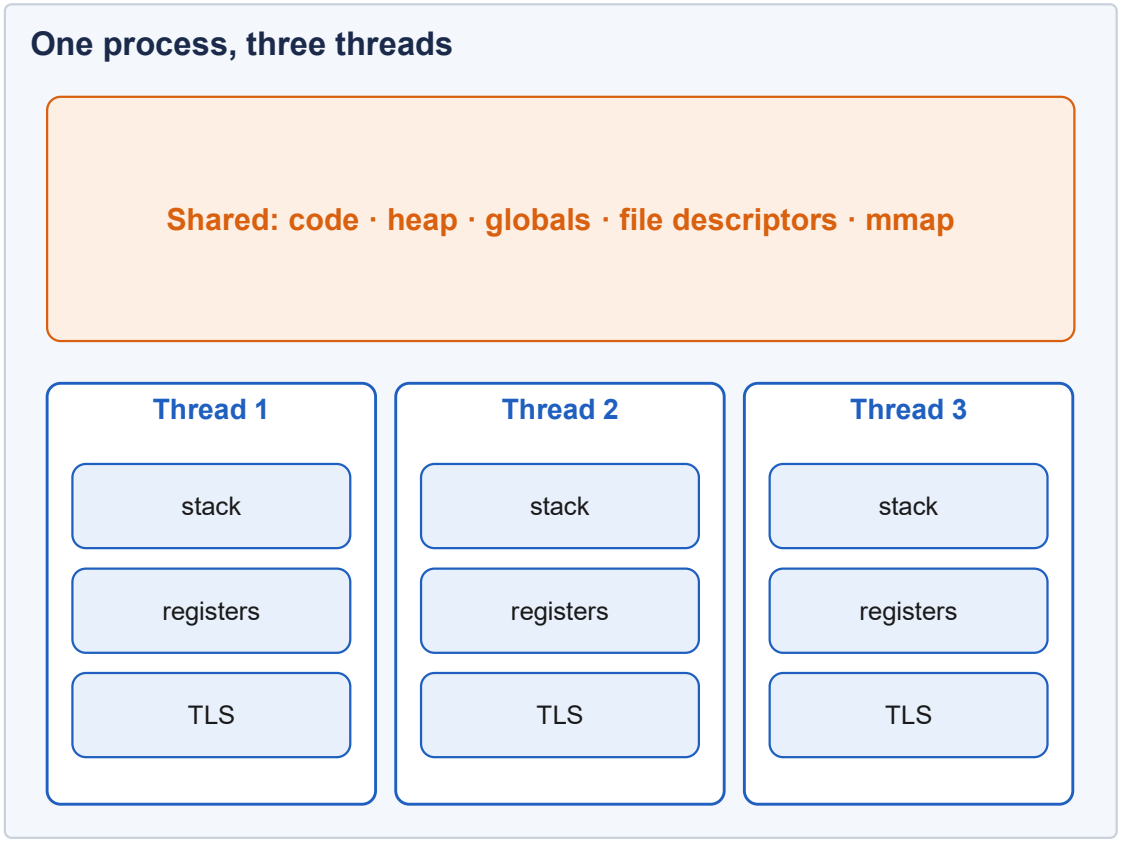
Many threads share data — first make it correct, then keep it responsive.

PART 2: CPU Performance and Scheduling

- **Week 6**

- **Why Do We Need Locks?**
  - Shared data → the counter++ race → atomics and their cost → memory ordering
- **Spinning and Sleeping Locks**
  - spinlock, qspinlock → mutex, rwsem, semaphore → futex → granularity, contention
- **Advanced Locking: RCU**
  - Readers without locks → read, copy, update, reclaim → the grace period
- **Concurrency Bugs**
  - Atomicity, order, deadlock → hidden ABBA → checkpoint: find the bugs
- **Tools for Concurrency Bugs**
  - lockdep, ThreadSanitizer → debugging hangs → perf lock, c2c, trace

Threads Share Everything — Except What You Make Private



1 Not shared

Local variables, thread_local, per-thread queues — no sync needed

2 Shared, read-only

Initialize before starting threads, then only read — no sync needed

3 Shared, mutable

Needs a lock, atomics, or single-owner hand-off

```
static thread_local char buf[256];  // C11
pthread_create(&t, NULL, fn, arg);
pthread_join(t, NULL);
```

Best synchronization is none: make data private or immutable whenever you can.

The Problem: counter++ Is Not One Step

```
long counter = 0;       // shared

void *worker(void *p) {
  for (int i = 0; i < 1000000; i++)
    counter++;           // load, add, store
  return NULL;
}
```

```
$ ./race    # 2 threads × 1,000,000
counter = 1263407
expected  2000000
```

Output is illustrative and changes every run.

Time	Thread A (CPU 0)	Thread B (CPU 1)	counter
t1	load r = 5		5
t2		load r = 5	5
t3	add r = 6		5
t4		add r = 6	5
t5	store 6		6
t6		**store 6**	**6 (lost update!)**

One core is not safe either: preemption can switch threads between load and store.

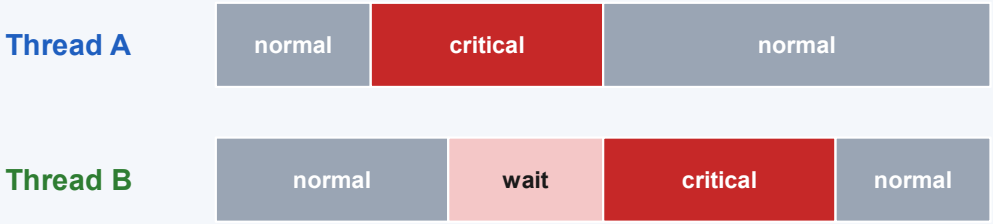
Three separate problems

- **Atomicity**: the read-modify-write must be indivisible
- **Visibility**: other cores must see the new value
- **Ordering**: other memory operations must not move across it

A race happens when two threads access the same data, at least one writes, and nothing orders the accesses.

Critical Sections and What a Lock Must Guarantee

Two threads, one lock




```
lock()  →  critical section  →  unlock()
```

- **Entry:** acquire the lock (may spin or sleep)
- **Critical section:** touch shared data — keep it short
- **Exit:** release; publish all writes to other cores

Critical section length directly limits scaling (Amdahl, Part 2).

1 Mutual exclusion

At most one thread inside the critical section

2 Progress

If nobody is inside, someone waiting gets in

3 Bounded waiting

No thread waits forever (fairness)

4 Performance

Cheap when uncontended; few cache-line moves

A lock turns concurrent access into ordered access — correctness first, then make the ordered part short.

Atomic Instructions: Arm LSE and x86

C11 — the same source on both CPUs

```
#include <stdatomic.h>
atomic_long counter;
atomic_fetch_add(&counter, 1);  // indivisible
```

The compiler emits one instruction

Arm `-march=armv8.1-a`

```
ldadd  x2, x1, [x0]
```

x86-64 `gcc -O2`

```
lock xadd  [rdi], rax
```

Operation	Meaning	Used for	Arm (Armv8.1 LSE)	x86-64
Fetch-and-add `atomic_fetch_add`	Read the value and add to it	**Atomic counter**	`ldadd x2, x1, [x0]`	`lock xadd [rdi], rax`
Compare-and-swap (CAS) `atomic_compare_exchange`	Change only if the value equals the expected one	**Lock implementation**	`cas x1, x2, [x0]`	`lock cmpxchg [rdi], rdx`
Swap (exchange) `atomic_exchange`	Replace the old value with a new one	**Spinlock**	`swp x2, x1, [x0]`	`xchg [rdi], rax`

Why CAS matters for the OS: only one CPU can win

```
atomic_int lock = 0;      // 0 = free, 1 = held
int expected = 0;
atomic_compare_exchange_strong(&lock, &expected, 1);
```

CPU 0: CAS 0 → 1 ✓ **wins: lock = 1, it owns the lock**

CPU 1: CAS 0 → 1 ✗ **fails:** sees 1, expected = 1 → retry or sleep

Run at the same time, **only one CAS changes 0 → 1** — the base of mutexes, spinlocks and lock-free data structures.

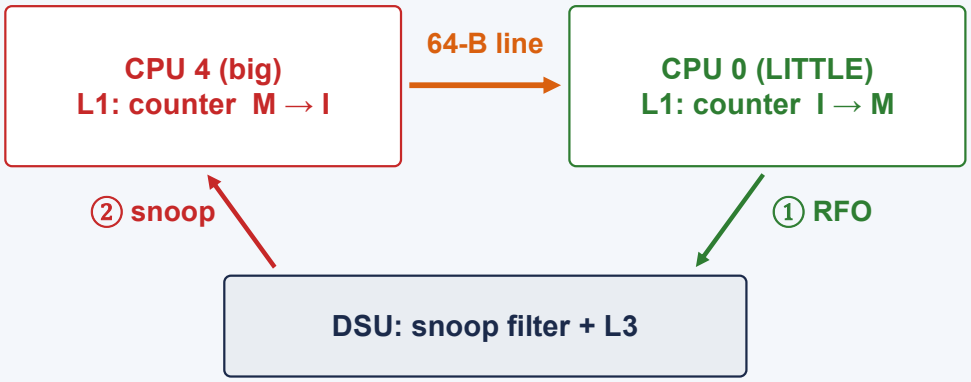
Atomics Are Not Free: What MESI Does on Every Add

Example: 4 cores, one shared counter

```
atomic_long counter;           // one 64-B cache line
// on CPU 4, 5, 0, 1: 1,000,000 times each
atomic_fetch_add(&counter, 1);
```

1. CPU 0 misses: the line is M in CPU 4
2. CPU 0 → DSU: RFO (read for ownership)
3. snoop filter → CPU 4: M → I, sends the line
4. CPU 0: I → M, adds 1 — next add on CPU 4: reverse

One add on CPU 0 (CPU 4 did the last add)



Case (Example, illustrative)	Where the line is	≈ per add
Plain add, 1 thread	M in own L1	**~1 ns**
Atomic add, 1 thread	M in own L1	**~2–5 ns**
Atomic add, 4 threads, same line	moves core → core every add	**~30–100 ns**
Per-CPU counters (own line each)	stays M in each L1	**~2–5 ns**

Cache Line Ping-Pong

Many CPUs modify the same line → **ownership keeps moving** between cores, and every add waits for it.

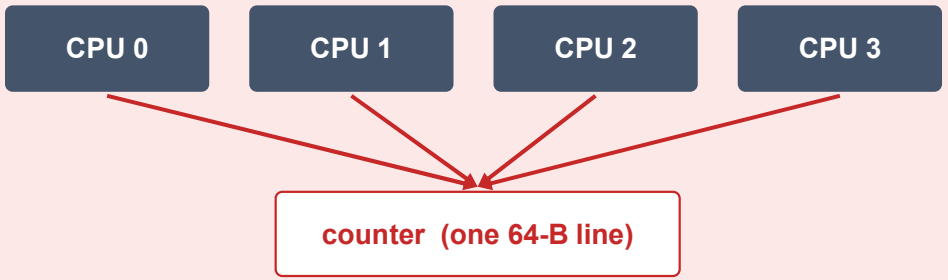
Example: 4 CPUs × 1 M adds = 4 M adds on one line → 4 M × ~50 ns ≈ **200 ms**.

Fix → per-CPU counters (next slide)

An atomic is cheap only while its line stays in your cache — **sharing turns every add into a cache-line transfer**.

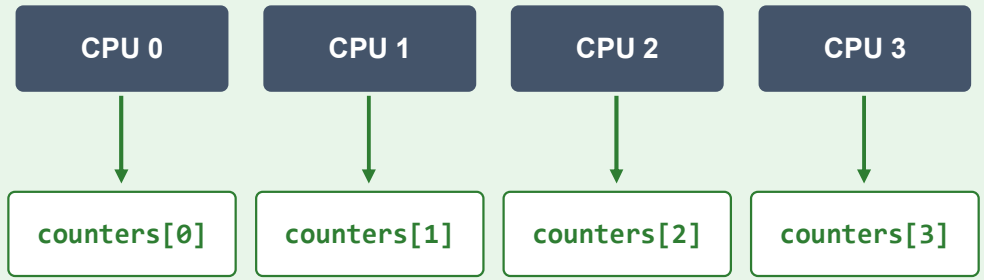
Avoiding Ping-Pong: Per-CPU Counters

✗ One shared counter: the line ping-pongs



ownership moves on every add

✓ Per-CPU counters: each CPU owns its line



one line each → stays M in its own L1, almost no transfer

Code (Example)

```
atomic_long counters[4];   // ✗ all 4 share one 64-B line

struct pcpu_cnt {
    _Alignas(64) atomic_long v;   // ✓ padded: one line each
};
struct pcpu_cnt counters[4];
atomic_fetch_add(&counters[cpu].v, 1);   // CPU i: own line
```

Example: 4 CPUs × 1 M adds

- **Shared line:** ≈ **200 ms** (4 M adds, one at a time)
- **Per-CPU:** 1 M × ~3 ns in parallel ≈ **3 ms** (≈ 60×)
- **Need the total?** read and sum all counters — only then
- **Linux:** `percpu_counter`, `this_cpu_inc()`

Give each CPU its own cache line — **add locally, sum only when someone asks**.

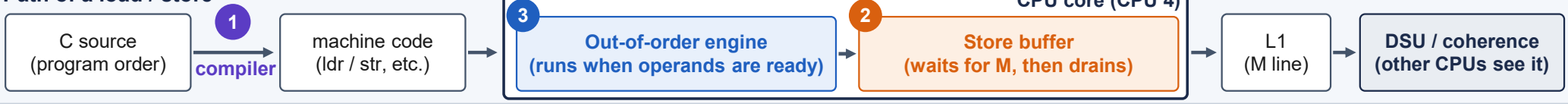
Memory Ordering: Where Reordering Happens

```
// Thread A on CPU 4
data = 42;    // (1) normal store
flag = 1;     // (2) normal store
```

```
// Thread B on CPU 0
while (flag == 0) ;  // spin
print(data);   // 42 or 0?
```

On Arm, B may print 0.
Who changed the order? Three places on the path below.

Path of a load / store



1 Compiler — before the CPU

The compiler can reorder independent memory operations and keep values in registers.

• Reorder stores — possible order

```
data = 42;
flag = 1;
```
→
```
flag = 1;
data = 42;
```

• Load hoisting — reads flag once!

```
while (flag==0)
  ;
```
→
```
r0 = flag;
while (r0==0) ;
```

• Store elimination — first store removed

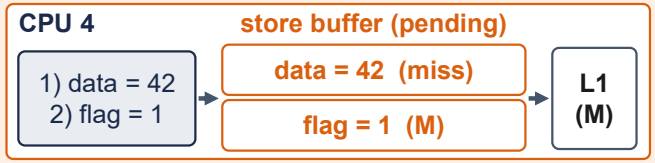
```
x = 1;
x = 2;
```
→
```
x = 2;
```

Allowed: same single-thread result (as-if rule)

2 Store buffer — after execution

A store has executed but stays in the store buffer until its cache line becomes M and can drain.

- A store completes quickly and goes into the per-core store buffer
- The core keeps running later instructions (flag = 1) while data = 42 is still buffered
- A store drains to L1 when its line is M — only then do other CPUs see it



x86: FIFO drain, stores stay in order. **Arm:** flag (M) may drain before data.

3 Out-of-order — inside the CPU

The CPU executes memory operations when their operands are ready, not strictly in program order.

• A load executes early if its address is ready

```
LDR X0, [A]   // cache miss (long)
LDR X1, [B]   // cache hit (short)
```

LDR [A] (waiting for memory)

LDR [B] ✓ early

• A store executes early past a pending load

```
LDR X0, [A]   // cache miss
STR X1, [B]   // address ready
```

LDR [A] (waiting)

STR [B] early → store buffer

Program order can change at three places before another core sees it: **compiler, store buffer, and out-of-order execution**.

Three Reorderings, by Example

1 Compiler reordering

The compiler can reorder memory operations and keep values in registers.

Source (what you wrote)

```
while (flag == 0)
  ;  // spin
print(data);
```

Compiled (-O2)

```
r = flag;
while (r == 0)
  ; // forever
print(data);
```

What happens?

- `flag` is a plain variable: the compiler assumes no other thread changes it
- It hoists the load out of the loop and keeps the value in a register
- **Result:** spins forever, even after another CPU sets `flag = 1`

Visualization



Fix

- **C11:** `atomic_load(&flag)`
- **Linux:** `READ_ONCE(flag)`
- Forces a real load on every iteration

2 Store buffer reordering

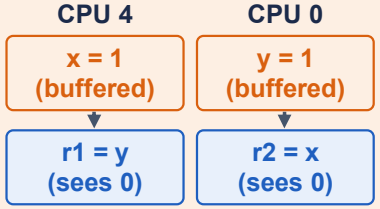
A store completes locally but stays in the store buffer and becomes visible to other cores later.

Example: Store Buffering (SB) litmus test

```
// CPU 4
x = 1;  // store
r1 = y; // load
```

```
// CPU 0
y = 1;  // store
r2 = x; // load
```

Initially x = y = 0. **Possible: r1 = 0, r2 = 0** — allowed on x86 and Arm



1. Stores wait in the buffers
2. Loads read the old 0 from cache
3. Stores drain later → r1 = r2 = 0

What happens?

- A later load passes an earlier store still in the buffer — even on x86 (TSO allows store → load)

Fix

- **Full barrier between them:** `smp_mb()`, DMB (Arm), MFENCE (x86)
- C11: seq_cst — **release / acquire is not enough**

3 Out-of-order loads

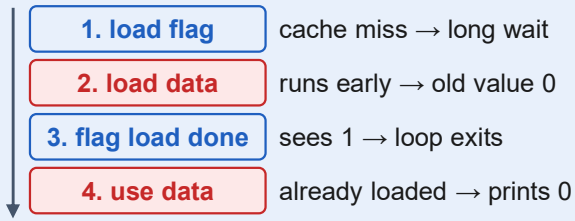
The CPU can run a later load before an earlier one that is not ready (e.g., a cache miss).

Example: message passing

```
// CPU 4 (producer)
data = 42;
smp_store_release(&flag, 1);
```

```
// CPU 0 (consumer)
while (READ_ONCE(flag) == 0) ;
print(data);     // may print 0
```

How it can happen on CPU 0



Fix

- **Acquire load:** `smp_load_acquire(&flag)`, LDAR (Arm), or DMB before data
- x86 keeps load → load order; **Arm needs this**

Each layer can reorder on its own — **a correct lock (acquire / release or stronger) prevents all three**.

Memory Models: x86-TSO vs. Arm (with Examples)

x86-TSO = Total Store Order

A relatively strong memory model.

- All cores see each core's stores in program order
- Stores become visible in one order (FIFO store buffer)
- Loads are not reordered with older loads / stores — **except store → load**
- **TSO ≈ program order + FIFO store buffer**

Arm = weaker memory model

A relaxed memory model.

- Loads and stores can be reordered, unless constrained
- A store may reach other cores at different times
- Only dependencies and barriers limit reordering
- **Software must ask for order:** acquire / release, DMB

Same code, different outcome

Message passing, no atomics / barriers

```
// Thread A
data = 42; // (1)
flag = 1;  // (2)
```

```
// Thread B
while (flag == 0);
use(data);  // ?
```

On x86: works (usually) — flag = 1, then data = 42
On Arm: may see data = 0
(In standard C: a data race → undefined behavior)

Which reorderings are allowed?

same-address order is always kept

Access order	x86 (TSO)	Arm (weak)	Example	Observed?
Load → Load	kept	allowed	`r1 = x; r2 = y;`	yes on Arm
Load → Store	kept	allowed	`r1 = x; y = 1;`	yes on Arm
Store → Store	kept	allowed	`x = 1; y = 1;`	yes on Arm
Store → Load	allowed	allowed	`x = 1; r1 = y;`	**yes on both**

Why the code can fail on Arm

1 Producer-side

flag = 1 can become visible before data = 42.

Store buffer (may drain out of order)

flag = 1 (visible first)
data = 42 (visible later)

2 Consumer-side

The load of data can run before the load of flag.

Out-of-order execution

load data (executes early → 0)
load flag (later, cache miss)

x86-TSO: typical execution (works)

CPU A	data = 42	flag = 1
CPU B	flag (1)	data (42)

A's stores become visible in program order, so B reads 42 after seeing flag = 1.

Arm: possible execution (can fail)

CPU A	flag = 1	data = 42
CPU B	flag (1)	data (0)

flag becomes visible first (or data is loaded early): B sees flag = 1 but reads data = 0.

Why do architectures differ?

x86: hardware pays

- Old software (since 1978) assumed strong order
- FIFO store buffer, load-order tracking → area, power
- every `lock` instruction is a full barrier

Arm: software pays

- Low power, small cores, scalability
- Asks for order only where needed: DMB, LDAR, STLR
- `ldadda` / `ldaddl` / `ldaddal`

x86 buys ordering with hardware; Arm makes you ask for it — portable code uses atomics with the right memory order.

Fixing It: Barriers and Acquire/Release

C11 (user space): release / acquire

```
// Producer (CPU 4)
data = 42;  // normal
atomic_store_explicit(
  &flag, 1,
  memory_order_release);
```

```
// Consumer (CPU 0)
while (
  atomic_load_explicit(
    &flag,
    memory_order_acquire)
  == 0) ;
print(data);  // 42
```

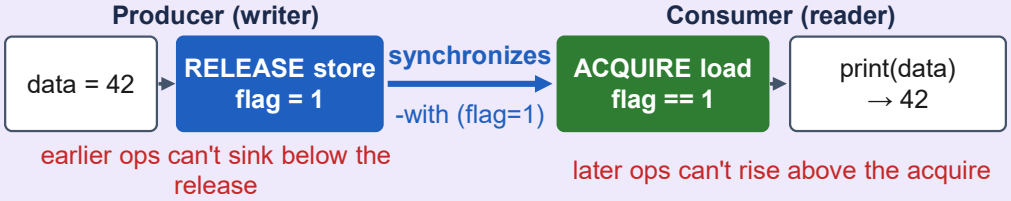
Linux kernel: release / acquire

```
// Producer (CPU 4)
data = 42;
smp_store_release(&flag, 1);
// Consumer (CPU 0)
while (!smp_load_acquire(&flag))
  cpu_relax(); // spin hint
print(data);  // 42
```

Resulting guarantees

- Acquire reads the release's value → **every write before the release is visible**
- Writes can't sink past the release; reads can't rise above the acquire
- Enough for message passing

What release and acquire do



Locks already do this

```
lock();      // acquire
shared = 42; // critical
unlock();    // release
```

A correct lock gives **mutual exclusion + memory ordering**

- `lock()` = acquire: later ops can't move before it
- `unlock()` = release: earlier ops can't move after it
- The next lock holder sees every write made before unlock()

Common tools	What it prevents	Notes
`READ_ONCE() / WRITE_ONCE()`	compiler caching / reordering (a real load, store)	no CPU ordering between locations
`atomic_load / store (acquire / release)`	acquire: later ops before · release: earlier ops after	use as a pair (C11)
`smp_load_acquire() / smp_store_release()`	the same, in the Linux kernel	Arm: LDAR / STLR; x86: plain mov
`smp_mb() · DMB · MFENCE`	full barrier, both directions (also store → load)	stronger, more expensive
`memory_order_seq_cst`	one global order for all seq_cst atomics	simplest; may cost more on Arm

Pair a release on the writer with an acquire on the reader — or just use a lock.

PART 2: CPU Performance and Scheduling

- **Week 6**

- Why Do We Need Locks?
  - Shared data → the counter++ race → atomics and their cost → memory ordering
- **Spinning and Sleeping Locks**
  - spinlock, qspinlock → mutex, rwsem, semaphore → futex → granularity, contention
- Advanced Locking: RCU
  - Readers without locks → read, copy, update, reclaim → the grace period
- Concurrency Bugs
  - Atomicity, order, deadlock → hidden ABBA → checkpoint: find the bugs
- Tools for Concurrency Bugs
  - lockdep, ThreadSanitizer → debugging hangs → perf lock, c2c, trace

Why the RX Queue Needs a Lock: A Broken Linked List

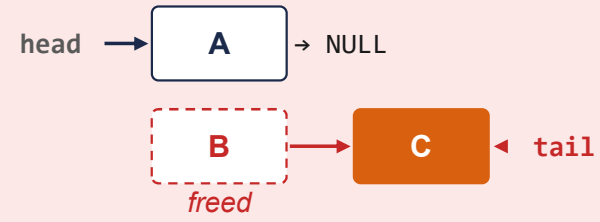


Two contexts share one RX queue: the driver thread **adds** buffers, the IRQ handler **removes** packets and passes them to TCP/IP. On different CPUs they run at the same time; on the same CPU the IRQ can interrupt the driver mid-update.

Start



After ③: the queue is broken



Driver thread (CPU 0): add C

```
① last = tail;      // B
```

```
③ last->next = C;  // freed B!
   tail = C;
```

IRQ handler (CPU 1): remove B

```
② A->next = NULL;
   tail = A;
   free(B);
```

Result: C is lost, tail points outside the list, freed memory was written → lost packets or a kernel crash.

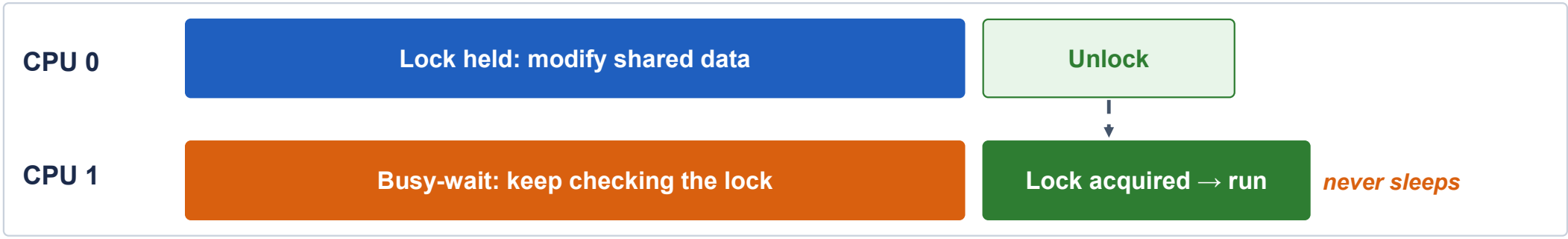
Fix: only one context may change the queue at a time → a **lock** (next slides).

Add and delete each look correct alone — **interleaved, they break the list**.

Spinning Locks vs. Sleeping Locks

Many CPUs can touch shared kernel data at once — a lock keeps it consistent. Two kinds:

	Spinning lock	Sleeping lock
Cannot get the lock →	keeps checking, again and again	sleeps until it is woken
CPU while waiting	**burns CPU**	**gives the CPU to another task**
Good for	very short critical sections	longer waits
Example	`spin_lock()`	`mutex_lock()`



A spinlock wins when the lock is held **very briefly** — a long wait just **wastes the CPU**.

Kernel Spinlock: Protecting the RX Queue

Driver thread (process context): add

```
static DEFINE_SPINLOCK(q_lock); // guards rx_queue
unsigned long flags;
spin_lock_irqsave(&q_lock, flags);   // + IRQs off
    list_add_tail(&pkt->node, &rx_queue);          critical section
spin_unlock_irqrestore(&q_lock, flags); // IRQs on
```

IRQ handler: remove

```
spin_lock(&q_lock);     // IRQs are already off here
    pkt = list_first_entry(&rx_queue, ...);
    list_del(&pkt->node);                          critical section
spin_unlock(&q_lock);
```

What `spin_lock_irqsave()` does

1. save the IRQ state in flags
2. IRQs off on this CPU only → the handler cannot cut in here
3. preemption off → the holder is not switched out
4. take the lock; if another CPU has it, spin

Why IRQs off? Only this CPU's IRQs go off — CPU 1 still takes the IRQ and runs the handler, which then spins on q_lock. Without it, an IRQ on CPU 0 would spin on a lock its own CPU holds — forever.

Rules

- **Hold it for microseconds** — only fast memory work
- **Never sleep while holding it** (functions that may sleep: p24)
- **Shared with an IRQ** → **`_irqsave`**; check the calling context and IRQ state

Variant (non-RT)	Shared with	Turns off (this CPU)
`spin_lock`	process context only	preemption
`spin_lock_bh`	softirq / tasklet	+ softirqs
`spin_lock_irqsave`	hard IRQ handler	+ hard IRQs (state saved)
`raw_spin_lock`	must spin, even on PREEMPT_RT	preemption; always spins

The lock makes add and delete take turns — IRQs off keeps the handler from cutting in on the same CPU.

Simple Spinlock: Why One Shared Word Does Not Scale

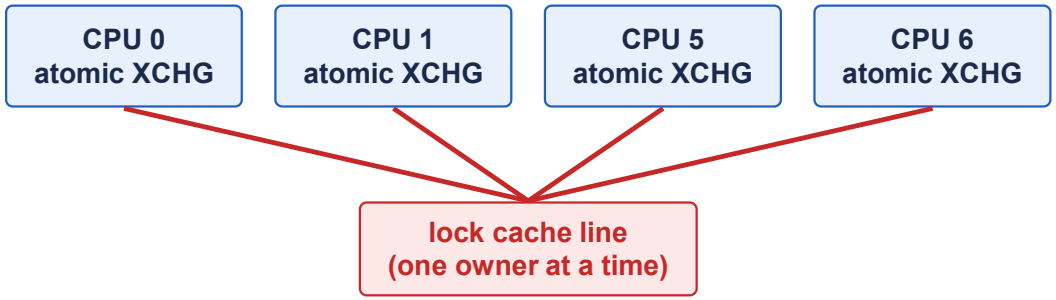
Test-and-set (TAS): every waiter repeatedly performs an atomic read-modify-write on the **same cache line**.

① Test-and-set: repeated atomic RMW

```
while (atomic_exchange(&lock, 1) == 1)
    ;   // each try asks for write ownership
/* critical section */
atomic_store(&lock, 0);   // unlock
```

Even while lock = 1, every failed exchange **still asks for exclusive (M) ownership** of the line. More CPUs → more coherence traffic.

② Cache-line ownership ping-pong



Ownership moves between cores; the atomics serialize.

When CPU 4 (the holder) unlocks:

1. **Store lock = 0**
   cached copies in the waiters are invalidated
2. **Waiters retry**
   many CPUs request ownership at once
3. **One CPU wins**
   its exchange sees 0; the others see 1
4. **Contention continues**
   the losers keep spinning on the same line

TAS creates coherence contention even before unlock. **TTAS cuts failed atomics; queue locks keep spinning local.**

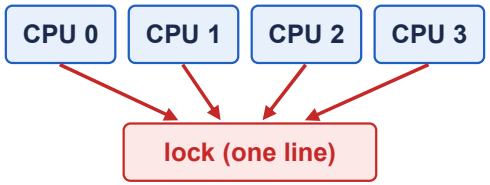
MCS Lock: Each Waiter Spins on Its Own Node

1. Why a simple spinlock scales poorly (test-and-set)

```
// 0 = free, 1 = taken
while (xchg(&lock, 1) == 1)
    ;  // RMW every time
atomic_store(&lock, 0);
```

Problem
- RMW on one shared word
- Its line keeps bouncing
- Unlock wakes all (herd)

Cache-line bouncing with many waiters

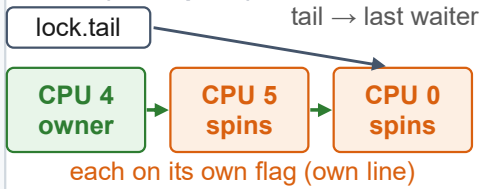


1. All CPUs try atomic_exchange()
2. The line moves to one CPU (M)
3. Other copies are invalidated
4. Next try → it moves again
5. Repeats until one succeeds

2. MCS lock: a queue + per-CPU spinning

```
struct mcs_node { *next; locked; }
mcs_lock(l, n):
  n->next = NULL; n->locked = 1;
  prev = xchg(&l->tail, n);
  if (prev) { prev->next = n;
    while (n->locked) relax(); }
                // own flag
mcs_unlock(l, n):
  if (!n->next) {
    if (cas(&l->tail,n,NULL)) return;
    while (!n->next) relax(); }
  n->next->locked = 0; // hand off
```

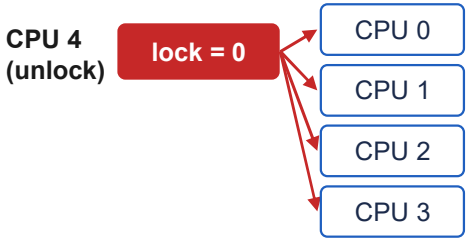
Queue (conceptual)



- Spin on **your own node**
- Unlock sets only the **next node's flag**
- Lock passes in FIFO order

3. Unlock: simple spinlock vs. MCS

Simple: thundering herd



All waiters wake and race for the same line

MCS: direct hand-off



Only the next waiter wakes — no global contention

4. Comparison

Aspect	Simple spinlock (TAS)	MCS lock
Each waiter spins on	the same lock variable	its own node's flag
Traffic while waiting	**grows with waiters (~N)**	**minimal (spins locally)**
Unlock	**wakes all (thundering herd)**	**wakes only the next**
Who gets the lock	whoever wins the race	**FIFO order (fair)**
More CPUs	more contention, slower	**scales much better**
Extra memory	none	one node per waiter

MCS does not remove spinning — it removes **contention on one shared cache line**: each waiter spins on its own node, and the lock is passed to the next.

Linux qspinlock: MCS, Made Small and Fast

Same API (e.g., RX queue):

```
spin_lock(&q_lock);  list_add(...);  spin_unlock(&q_lock);
```

Same interface as a simple spinlock — very different inside and in scalability.

Where qspinlock is used

- The kernel's general spinlock (`spinlock_t`)
- Short critical sections that cannot sleep (IRQs / preemption off)
- Cheap when uncontended, scalable under contention

One 32-bit lock word (typical 64-bit kernel)

locked (bits 0–7) held or not	pending (bits 8–15) one waiter on deck	tail (bits 16–31) last waiter in the MCS queue
• 0: free • 1: held	• 0: no pending waiter • 1: a waiter is pending	• CPU number + per-CPU node index • 0: no queue

Why 32-bit?

- Each lock stays 4 bytes — the kernel has millions of locks
- State as bits, not full pointers
- MCS nodes are per-CPU, not in the lock

Three acquisition paths, depending on contention

1 No contention: fast path (one atomic CAS)

```
lock(l):
  val = 0;
  if (try_cmpxchg_acquire(
        &l->val, &val, 1))
    return;  // success
  slowpath(l, val);
```

word = 0 → **locked = 1**

✓ one CAS
✓ no queue

2 One waiter: pending path (light contention)

1. The first waiter sets pending = 1
2. It spins on the lock word (no queue node)
3. On unlock, the pending waiter takes the lock

CPU 4 (owner) → locked = 1, pending = 1 ← **CPU 5 (pending)**

3 Many waiters: MCS queue (heavy contention)

1. More waiters join the queue via tail (xchg)
2. Only the queue head watches the lock word
3. The others spin on their own MCS node

CPU 4 owner → **CPU 5 head** → **CPU 0 node** → **CPU 1 node**

Comparison: simple spinlock vs. Linux qspinlock

Aspect	Simple spinlock (TTAS)	Linux qspinlock
Uncontended acquire	one atomic (xchg / CAS)	**one CAS (fast path)**
Waiting location	the same lock word	**pending: lock word · queue: own node**
Unlock	wakes all waiters (herd)	**pending or next waiter only**
Coherence traffic	grows with waiters (~N)	**mostly owner ↔ next**
Memory per lock	4 bytes	**4 bytes (+ per-CPU nodes)**

Key takeaways

1. **Hybrid:** one CAS when free, pending for light, MCS queue for heavy contention
2. **Small:** all state fits in 32 bits (4 bytes)
3. **Scalable:** most waiters spin on their own nodes — no thundering herd
4. **Same API,** better performance, smaller memory

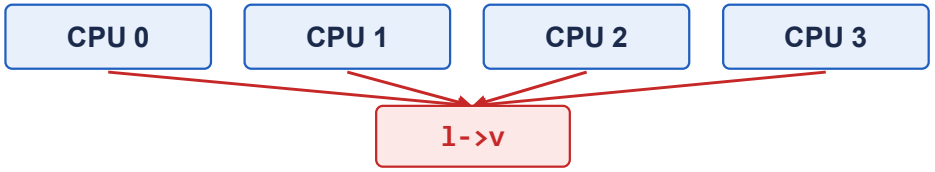
In Code: A Simple Spinlock vs. qspinlock

Used the same way on the RX queue: `lock(&q_lock); list_add(&pkt->node, &rxq); unlock(&q_lock);`

Simple spinlock (hand-made, C11 atomics)

```
void slock(slock_t *l) {
  while (atomic_exchange(&l->v, 1))  // take it?
    while (atomic_load(&l->v))       // spin on the
      ;                              //  same word
}
void sunlock(slock_t *l) {
  atomic_store(&l->v, 0);   // all waiters miss
}
```

Problem: everyone hits one variable



All read the same l->v. On unlock all see the change and try atomic_exchange() at once → one wins, the rest wait again. More waiters → more coherence traffic.

qspinlock (Linux spinlock_t, simplified)

```
void queued_spin_lock(struct qspinlock *lock) {
  int val = 0;
  if (atomic_try_cmpxchg_acquire(&lock->val,
                &val, _Q_LOCKED_VAL))
    return;                        // (1) fast path
  queued_spin_lock_slowpath(lock, val); // (2)
}
// unlock: smp_store_release(&lock->locked, 0)
```

(1) Fast path — lock free: a CAS. If the value is 0 → set _Q_LOCKED_VAL, success; if not 0 → no change, failure. A free lock costs **one atomic CAS**

(2) Slow path — lock taken: uses **`locked`** (held?), **`pending`** (a waiter on deck), **`tail`** (last MCS node). Light contention → pending path; heavy → MCS queue, each CPU waits on its own node. The queue head still checks the shared lock word.

Same shape — lock, work, unlock — but **qspinlock decides where waiters spin**.

qspinlock Still Busy-Waits — cpu_relax() Is Not Sleeping

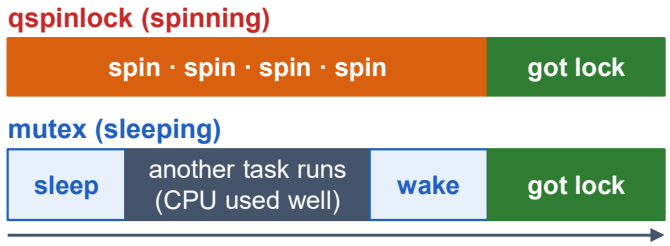
Pending or in the MCS queue, a qspinlock waiter **keeps running on its CPU** and checks the lock — it **never calls schedule() or sleeps**.

1. Conceptual wait loop in qspinlock

```
// in the MCS queue (or on pending)
while (READ_ONCE(node->locked))
    cpu_relax();   // still spinning
```

- `cpu_relax()` does **not** stop spinning or call the scheduler
- **It only hints: this is a spin-wait loop**

2. What CPU 1 does while it waits



3. Why cpu_relax() if still spinning?

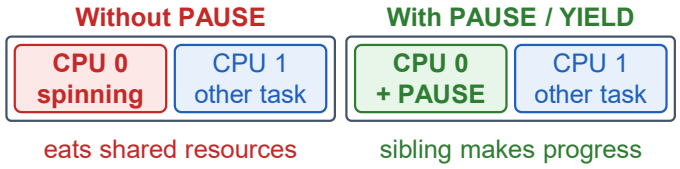
1. **Smaller pipeline penalty**
   tells the CPU it is a spin loop → less pipeline / memory-ordering cost on loop exit
2. **A good SMT sibling**
   on hyper-threaded cores, PAUSE / YIELD frees resources for the sibling logical CPU
3. **Lower power**
   less execution pressure and power while waiting
4. **Still spins — no scheduling**
   a hardware hint, not a call to schedule(); the task keeps running

4. What cpu_relax() is (architecture examples)

```
x86:   asm volatile("pause");  // PAUSE
arm64: asm volatile("yield");  // YIELD
```

Both are spin-wait hints — they help the SMT sibling make progress and may lower power.

5. Effect on a hyper-threaded (SMT) core



6. Comparison	Plain spin loop	With cpu_relax()	schedule() (e.g., mutex)
Spins? Calls schedule()?	spins · no	spins (with a hint) · no	no — blocks and sleeps · yes
CPU while waiting	**fully used**	**used, with less pressure**	**given to another task**
OK with IRQs / preemption off?	yes	yes	**no — must be able to sleep**
Suitable for	works, but wasteful	**short busy-waits (recommended)**	long waits, blocking work

qspinlock is still a **busy-wait lock** — cpu_relax() does not make it sleep; it makes spinning friendlier to the CPU.

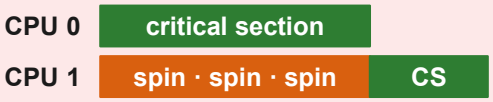
Sleeping Locks: mutex, rw_semaphore, semaphore (vs. spinlock)

1. What are they, and how do they wait?

Primitive	How waiters wait	Max inside	Sleeps?	Typical use	Key point
`spinlock`	spin (busy-wait)	1	no	very short, IRQ / atomic context	never sleeps; small and scalable (qspinlock)
`mutex`	spin briefly, then sleep	1	yes	general mutual exclusion (process ctx)	optimistic spin first; sleeps for long waits
`rw_semaphore`	sleep (readers / writers)	many readers or 1 writer	yes	read-mostly data, exclusive updates	readers share; writers are exclusive
`semaphore`	sleep	up to N	yes	limit access to N resources	counting, no owner (pools, device limits)
`RCU`	readers never wait	all readers	reader: no	read-mostly data structures	wait-free readers; writers wait a grace period

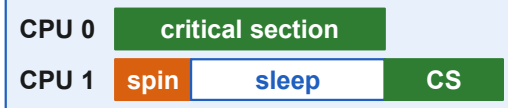
2. How each lock behaves when contended

spinlock — always busy-wait



keeps checking the lock; never calls schedule()

mutex — spin briefly, then sleep

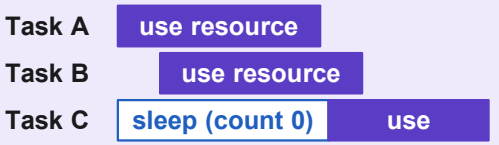


spins while the owner runs; a long wait → sleeps, woken on unlock

rw_semaphore — readers or 1 writer



semaphore — up to N users (N = 2)



extra tasks sleep until count > 0

3. Operations that may sleep

- `mutex_lock()` sleeps if the lock is held
- `kmalloc(GFP_KERNEL)` may reclaim memory and sleep
- `copy_to_user()` page fault → waits for disk I/O

⚠ **A spinlock turns preemption off** (atomic context). Sleeping inside → **BUG: sleeping function called from invalid context** or deadlock. Use GFP_ATOMIC or call it outside.

4. When to use which lock?

Scenario	Lock
Very short, atomic / IRQ context	**spinlock**
Mutual exclusion, process context	**mutex**
Read-mostly, exclusive updates	**rwsem / RCU**
Limit to N identical resources	**semaphore**
Read-mostly, very many readers	**RCU**

5. Key takeaways

1. **spinlock** busy-waits; never sleeps
2. **mutex, rwsem, semaphore** sleep and give the CPU away
3. Choose by: **can you sleep? read / write mix? how long?**
4. **Never sleep while holding a spinlock**

mutex and rw_semaphore, by Example

Both **mutex** and **rw_semaphore** are **sleeping locks**: contended waiters **may sleep** (some spin first), freeing the CPU for other tasks.

1 mutex: one task at a time (mutual exclusion)

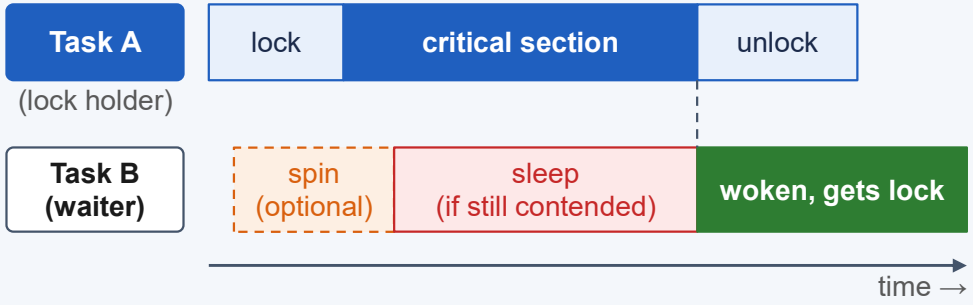
```
struct mutex lock;

mutex_lock(&lock);
/* critical section */
mutex_unlock(&lock);
```

Key points

- Only one task holds the lock
- Contended: a waiter may **spin** briefly (optimistic spinning), then **sleep**
- General mutual exclusion in process context

Execution timeline (two tasks contending)



2 rw_semaphore: many readers OR one writer

```
struct rw_semaphore rwsem;

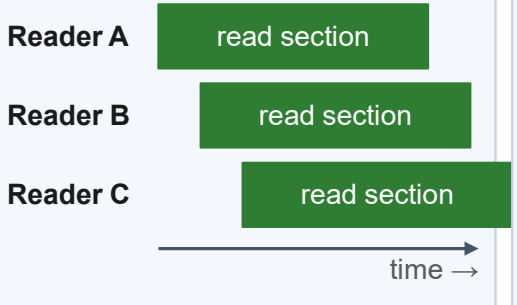
/* reader */
down_read(&rwsem);
/* read data */
up_read(&rwsem);

/* writer */
down_write(&rwsem);
/* modify data */
up_write(&rwsem);
```

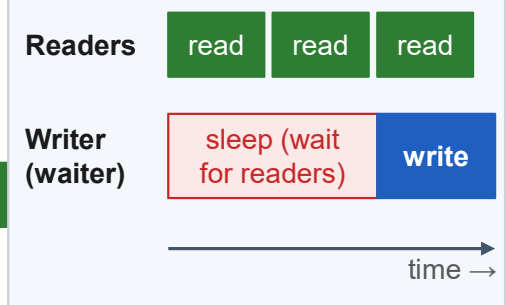
Key points

- Many readers at once
- Writers exclusive: wait for readers
- Waiters may sleep
- Read-mostly (mmap_lock)

Readers run in parallel



A writer waits for all readers



Exclusive access → **mutex**. Many readers, rare writers → **rw_semaphore** (or RCU, Part 3).

semaphore and RCU — Behavior and Examples

A **semaphore** is a **sleeping lock** for **up to N users**. **RCU** lets readers proceed almost without waiting; writers publish a new version.

3 semaphore: up to N concurrent users

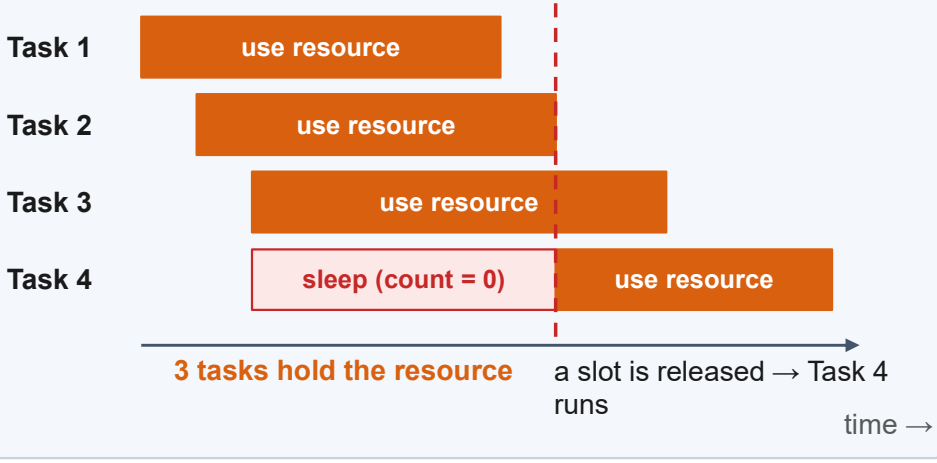
```
struct semaphore sem;
sema_init(&sem, 3); // N = 3

down(&sem);  // take one
/* use the resource */
up(&sem);    // give it back
```

Key points

- Up to N tasks inside at once
- Count 0 → waiters **sleep** (wait queue)
- Resource pools, device limits
- Plain mutual exclusion: use a **mutex**

Example: semaphore with count = 3



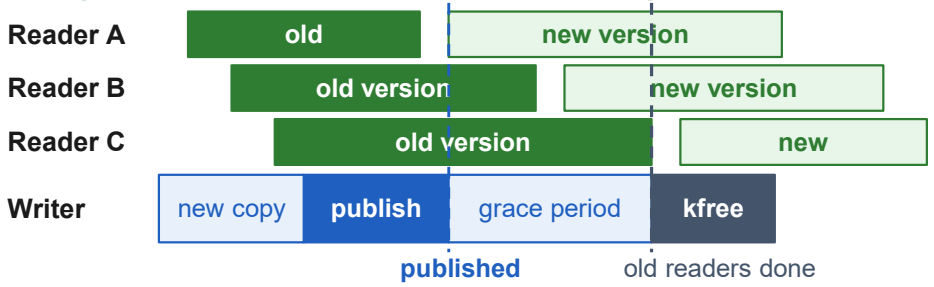
4 RCU: fast readers, writers publish a new version

```
/* reader: never waits */
rcu_read_lock();
p = rcu_dereference(gp);
/* read shared data */
rcu_read_unlock();
```

```
/* writer */
new = kmalloc(...); /* init */
rcu_assign_pointer(gp, new);
synchronize_rcu(); /* wait */
kfree(old);  /* now safe */
```

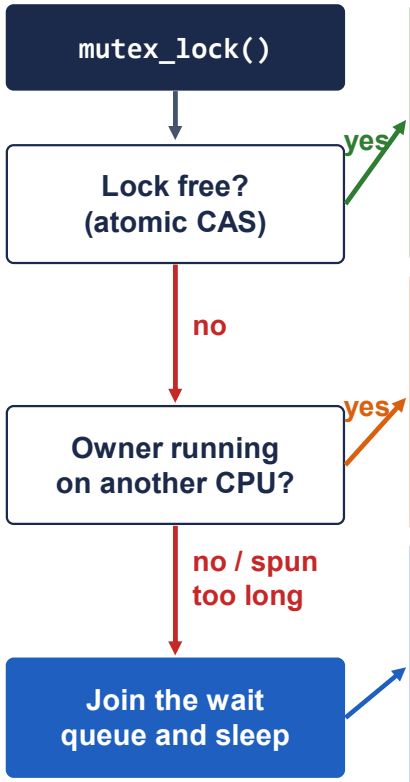
- Readers do **not** block and (non-RT kernel) must not sleep — very fast
- Writers do **not** sync with readers: publish a new version, wait a grace period, free the old
- Read-mostly structures: dentry cache, routing tables, RCU lists

Example: readers continue while the writer updates



Kernel mutex_lock(): Fast Path, Optimistic Spin, Slow Path

A kernel mutex is a **sleeping lock**. Contended, it **spins briefly** if the owner is running; otherwise it **sleeps on a wait queue**.



1 Fast path: lock acquired at once

How it works

- The lock is free
- Acquire it atomically (cmpxchg)
- No spinning, no sleeping — the fastest path

```
if (!cmpxchg(&owner, 0, me))
    return;  /* got the lock */
/* else: try the next paths */
```

No contention



2 Optimistic spinning: spin briefly — the owner may release soon

How it works

- Lock held, but the owner runs on another CPU
- Spin a short time; take it when released
- Avoids sleep + wake-up cost for short waits

```
while (owner_running(l) &&
       !need_resched()) {
  if (try_acquire(l)) return;
  cpu_relax();
}
```

Owner on another CPU, short CS



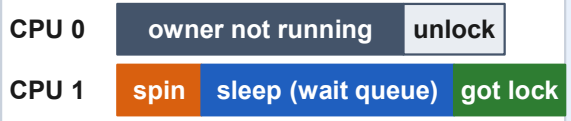
3 Slow path: join the wait queue and sleep — woken when the lock is released

How it works

- Owner sleeping or preempted, or the wait is long
- Add the task to the mutex wait queue; schedule()
- Woken on unlock → try to acquire again

```
add_to_wait_queue(l, me);
schedule();      /* sleep */
/* woken up */
if (try_acquire(l)) return;
/* else keep waiting */
```

Owner sleeps, or a long wait



A kernel mutex **spins only while it is likely to pay off**; if the owner is not running or the wait is long, it sleeps so the CPU can run other tasks.

Two Mutexes in Linux: Kernel mutex vs. pthread_mutex

	Kernel mutex	User-space mutex
API	`mutex_lock()`	`pthread_mutex_lock()`
Used in	kernel space	user space
Fast path	atomic CAS	atomic CAS
Middle step	optimistic spinning	may spin, depending on the implementation
Slow path	sleep on the kernel wait queue	sleep via futex(FUTEX_WAIT)
Uses futex?	**no**	**yes (usual Linux implementation)**

Kernel mutex (previous slide)

Kernel code → mutex_lock() → CAS / spin / sleep on the mutex's wait queue. Never a system call — it is already in the kernel.

pthread_mutex (next 3 slides)

Thread → pthread_mutex_lock() → CAS in user mode; only on contention → futex() system call → sleep in the kernel.



Same idea, two implementations: **the kernel mutex is not built on futex**.

Atomic CAS in User Space: Lock Without a System Call

CAS is a CPU instruction available in user mode — the uncontended fast path does **not** need to enter the kernel.

① Compare-and-swap: read → compare → update

```
int lock = 0;       // 0 = free, 1 = held
int expected = 0;
bool ok = atomic_compare_exchange_strong(
              &lock, &expected, 1);
```

CAS succeeds: 0 → 1
lock acquired

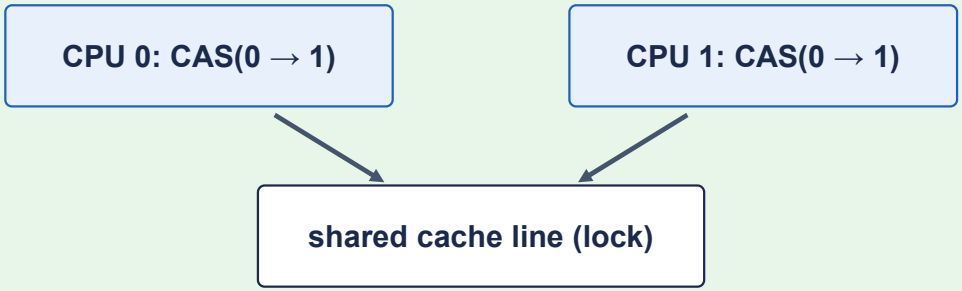
CAS fails: it is already 1
held — expected ← 1

② Arm64 LSE: one CAS instruction

```
mov   w1, #0          // expected
mov   w2, #1          // desired
casa  w1, w2, [x0]    // atomic compare + update
cmp   w1, #0          // old value 0 → success
```

x86: `lock cmpxchg [rdi], edx` (expected in eax) — also one instruction

③ Two CPUs racing for the same lock



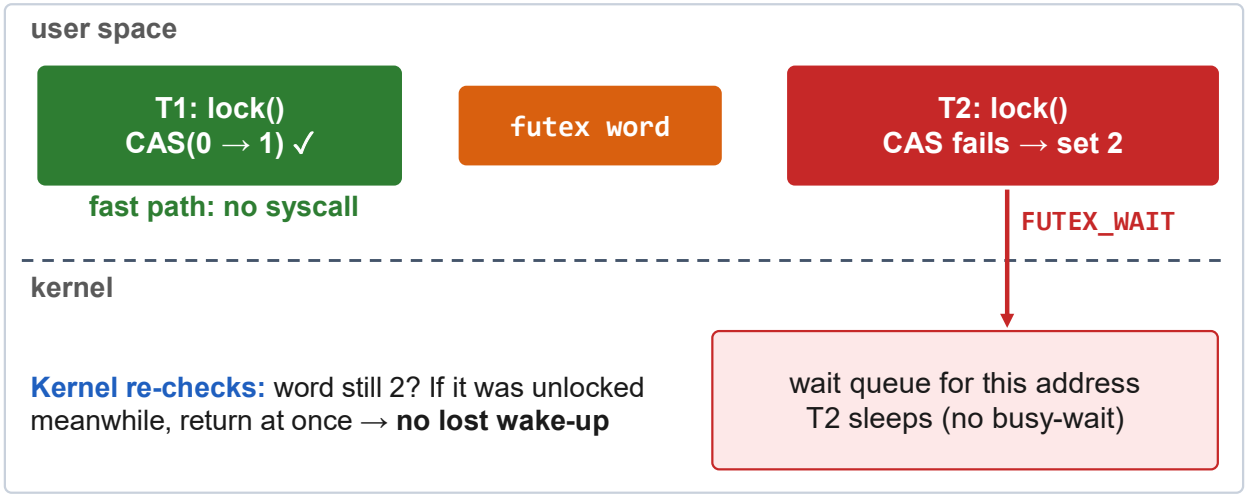
Atomic hardware + cache coherence
→ only ONE CPU changes 0 to 1

No syscall ≠ no hardware cost: cache-line ownership still moves between cores on every contended CAS.

CAS enables a **user-space fast path**; futex enters the kernel only when **waiting or waking** is needed.

futex (Fast Userspace Mutex): Fast Path and Slow Path

pthread_mutex_lock() in user space: **no contention** → one atomic. **Contention** → enter the kernel only to sleep (FUTEX_WAIT) and wake (FUTEX_WAKE).



Conceptual lock()

```
void lock(int *w) {
  if (CAS(w, 0, 1)) return;      // fast
  while (atomic_exchange(w, 2))  // mark 2
    futex_wait(w, 2);  // sleep if *w==2
}
```

Futex word	Meaning
0	unlocked
1	locked, no waiters marked
2	locked, waiters may exist

Note on 0 / 1 / 2: futex itself does not define these values — they are the convention of this simple mutex. glibc's pthread_mutex uses more states depending on the mutex type. And 2 means waiters **may** exist — not that a thread is surely asleep.

The kernel is entered **only when a thread must sleep** — and it re-checks the word so no wake-up is lost.

futex: Unlock, Wake-up and the Cost of Contention

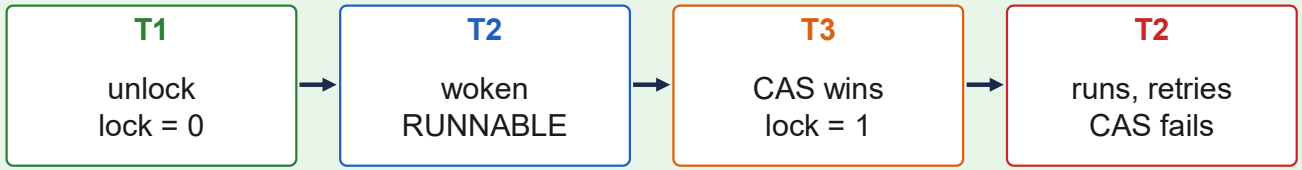
FUTEX_WAKE makes a waiter runnable — it does **not** transfer mutex ownership or guarantee immediate execution.

① Unlock: release, then wake

```
void unlock(int *w) {
  if (atomic_exchange(w, 0) == 2)
    futex_wake(w, 1);
}
```

Old value 2 → waiters may exist:
wake at most one. Old 1 → no syscall.

② Wake-up is NOT lock hand-off



Another thread may take the lock before the woken waiter runs.

③ Thread state transition

SLEEPING → RUNNABLE → RUNNING → retry CAS

The scheduler decides when the waiter runs

Runnable ≠ running: a wake-up may add scheduling delay.

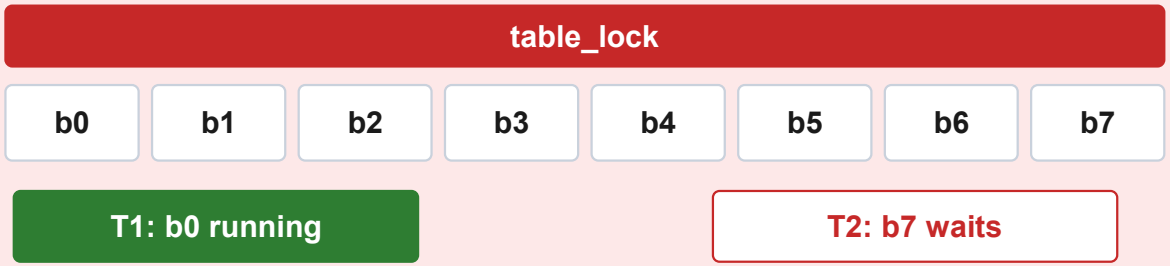
④ Contention costs when a waiter actually sleeps

- **1. System calls** FUTEX_WAIT / WAKE
- **2. Sleep / wake** wait-queue operations
- **3. Context switch** the CPU changes task
- **4. Scheduling delay** runnable, not running

Uncontended: **user-space atomic fast path**. Contended: **kernel wait / wake + possible switch and scheduling costs**.

Lock Granularity: One Big Lock vs. Lock Striping

COARSE: one mutex protects all 8 buckets

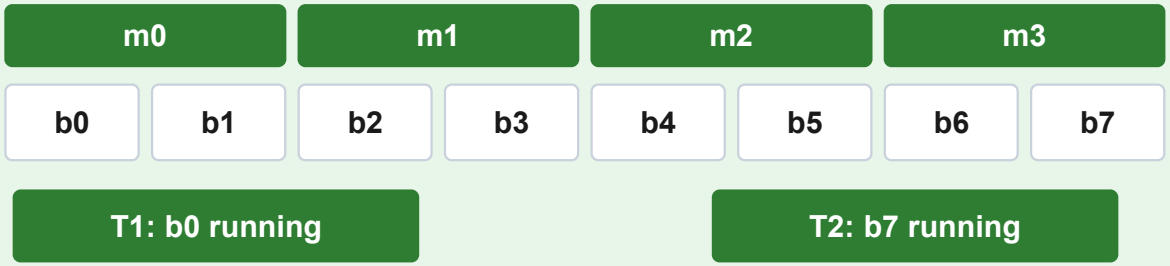


One shared mutex

```
pthread_mutex_lock(&table_lock);
modify_bucket(bucket);
pthread_mutex_unlock(&table_lock);
```

Any two updates wait for each other.

FINE: four mutexes protect 2 buckets each



Lock chosen by bucket ID

```
int id = bucket / 2;   // 0..3
pthread_mutex_lock(&locks[id]);
modify_bucket(bucket);
pthread_mutex_unlock(&locks[id]);
```

Different stripes run in parallel.

Trade-off: fewer locks = simpler, more contention | more locks = more parallelism, more complexity. *(Buckets independent; shared metadata needs its own sync.)*

Independent work should use **different locks** — not simply more locks.

Fine-grained Locks: Parallelism vs. Deadlock

Trade-offs: one big lock vs. lock striping

Aspect	One big lock	Lock striping
Parallelism	low	higher for independent buckets
Contention	can be high	can be reduced
Complexity, memory	low	more locks to manage
Deadlock risk	lower	higher with multiple locks

When fine-grained locking does NOT help

- All threads update b0 → all contend on m0
- One operation needs b0 and b7 → two locks
- More locks → more memory and lock operations

Deadlock: opposite lock acquisition order

```
T1: lock(m0)  ✓
    lock(m3)  → WAIT
```

```
T2: lock(m3)  ✓
    lock(m0)  → WAIT
```

T1 holds m0 ⇄ **T2 holds m3**

T1 waits for m3; T2 waits for m0 → circular wait

Fix: use ONE global lock acquisition order

```
// T1 and T2 both:
lock(m0); lock(m3);
... ;
unlock(m3); unlock(m0);
```

Both threads take m0 → m3 (unlock in reverse). The circular wait cannot form.



Split locks for parallelism — but **acquire multiple locks in one consistent global order**.

Amdahl's Law: Why Time Under a Lock Limits Scaling

Speedup = 1 / (s + (1 − s) / N)

s = serial fraction (cannot run in parallel) **N** = cores

Example: s = 25%, N = 4 → 1 / (0.25 + 0.75/4) ≈ **2.29×, not 4×**

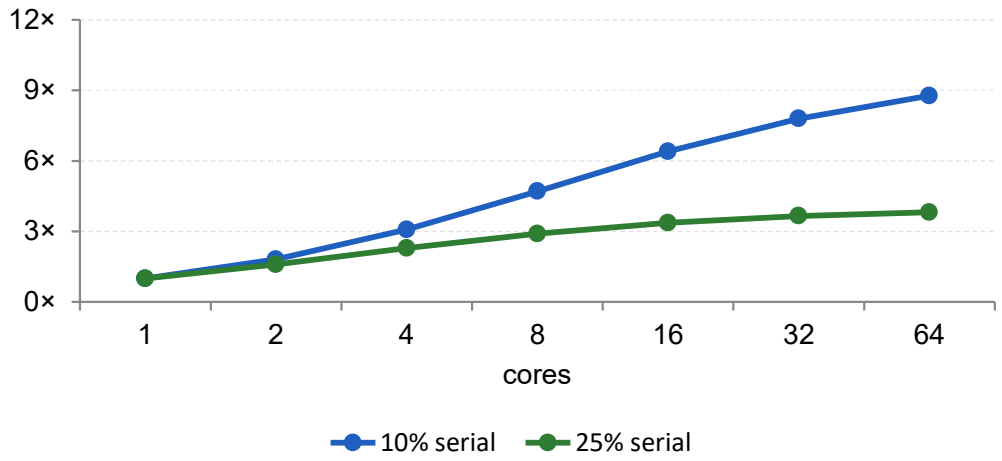
16 cores: 25% → 3.37×, 10% → 6.40× — cutting serial ≈ 1.9× faster

Why does a lock add serial time?

4 threads, one mutex: the code inside runs one thread at a time — other threads wait even with idle cores.

(Code under different locks can still run in parallel; 10% / 25% are simplified examples.)

Theoretical speedup vs. number of cores



Shrink the serial part first

✗ Before: lock held 11 ms

```
lock(&m);
prepare_data();    // 10 ms
update_shared();   // 1 ms
unlock(&m);
```

✓ After: lock held 1 ms

```
prepare_data(); // no lock
lock(&m);
update_shared();  // 1 ms
unlock(&m);
```

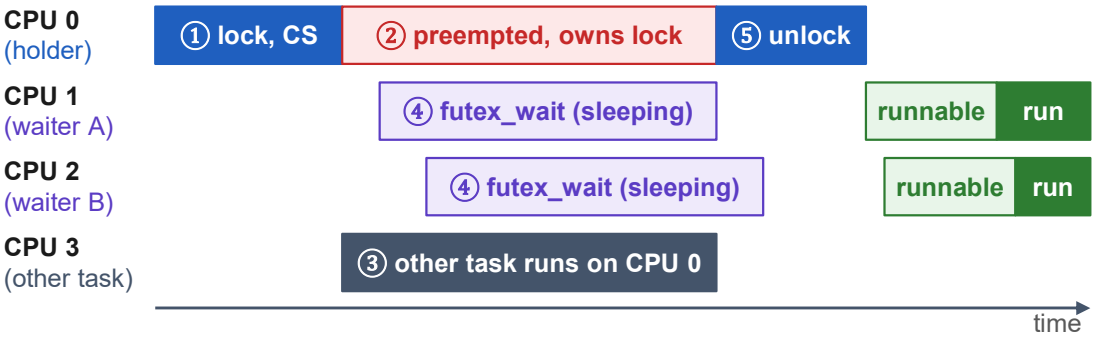
Q: 25% of the work is serialized by a lock. Can 100 cores give more than 4×?

A: No — under Amdahl the limit is 1 / s = **4×**.

Shrink the serial part first: **less work under the lock beats more cores**.

Lock Contention Meets the Scheduler

Lock holder preemption: a thread can hold a lock without running



1. Holder takes the mutex and runs its critical section
2. Holder is preempted (e.g., time slice ends) while holding it
3. Another task runs: the holder has no CPU, but still owns the mutex
4. Waiters cannot get the lock → they sleep in futex_wait()
5. Only after the holder runs again and unlocks can a waiter wake

1 Wake-up latency: wake-up ≠ immediate execution

Sleeping (in futex_wait) → Runnable (in run queue) → Running (on a CPU)

futex_wake() → run queue → CPU: the gap is **wake-up / scheduling latency**

2 Holder preemption (and priority inversion)

High-prio thread (waits for lock) → Low-prio holder (preempted) → Other task (medium-prio)

Medium-prio task preempts the holder → high-prio waiter stalls (**priority inversion**)

3 Lock convoys: once slow, many pay the cost

T1 holds lock → T2 wake + run → T3 wake + run → T4 wake + run

Every hand-off adds wake-up + scheduling delay → **stays slow**

Under contention, lock performance depends on:

Critical section length + Wake-up latency + Holder's sched. delay + Contention (convoy, …)

Mitigations

1. **Keep critical sections short** (no I/O or IPC under a lock)
2. **Consider the scheduling policy** of lock-holding service threads
3. **Use an adaptive mutex** (spin briefly, then sleep)

Adaptive mutex behavior

✓ **Holder runs, unlocks soon:** spin → avoid sleep / wake

spin | lock acquired

✗ **Holder preempted / long hold:** spinning wastes CPU → sleep

spin … | sleep (futex_wait)

Preemption does not release the mutex — **everyone waits for a thread that is not even running**.

PART 2: CPU Performance and Scheduling

- **Week 6**

- Why Do We Need Locks?
  - Shared data → the counter++ race → atomics and their cost → memory ordering
- Spinning and Sleeping Locks
  - spinlock, qspinlock → mutex, rwsem, semaphore → futex → granularity, contention
- **Advanced Locking: RCU**
  - Readers without locks → read, copy, update, reclaim → the grace period
- Concurrency Bugs
  - Atomicity, order, deadlock → hidden ABBA → checkpoint: find the bugs
- Tools for Concurrency Bugs
  - lockdep, ThreadSanitizer → debugging hangs → perf lock, c2c, trace

Readers Run Together in rw_semaphore — So Why RCU?

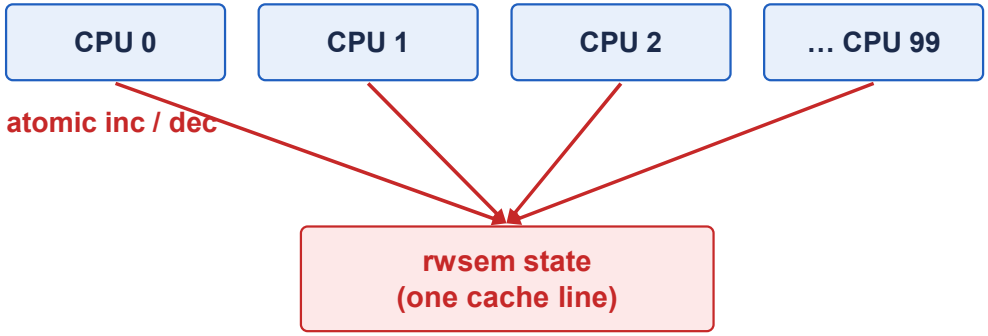
rw_semaphore already lets many readers into the critical section at once. So why RCU?
Readers that run **together** are not the same as readers that run **without synchronization cost**.

Example: 100 CPUs read one routing table

```
down_read(&rwsem);    // reader count++ (atomic)
/* read the routing table */
up_read(&rwsem);      // reader count-- (atomic)
```

✓ All 100 readers can be inside at the same time.

✗ **Hidden cost:** every down_read() / up_read() atomically updates the **shared lock state** — so its cache line moves between CPUs (p9). More readers → more contention.



And when a writer arrives: it must wait until **every reader has left**, and while it holds the lock, **new readers cannot enter**.

rw_semaphore makes readers **concurrent** — not **free**.

Reads Do Not Move Cache Lines — Writes Do

The routing table **is** shared data. What matters is whether readers only **read** it — or also **write** shared memory.

✗ rw_semaphore reader: writes the lock word

```
down_read(&rwsem);    // atomic write: count++
up_read(&rwsem);      // atomic write: count--
```

CPU 0	CPU 1	CPU 2	CPU 3
count: M	count: I	count: I	count: I

→ the next reader on CPU 1 must take the line to M again

- To write, the line must be M → other copies are invalidated
- So the count line moves from CPU to CPU (p9)
- The data is only read — the lock variable creates the writes

✓ RCU reader: loads only

```
rcu_read_lock();  p = rcu_dereference(gp);
read_data(p);     rcu_read_unlock();   // all loads
```

CPU 0	CPU 1	CPU 2	CPU 3
gp, table: S	gp, table: S	gp, table: S	gp, table: S

→ after the first miss, every CPU hits in its own L1

- rcu_read_lock() writes no shared memory (preemption off, or a per-task counter)
- rcu_dereference() and the reads are loads → lines stay S on all CPUs
- Nothing is invalidated → no coherence traffic

Only the writer moves a line: `rcu_assign_pointer(gp, new)` — one store, and writers are rare. "No shared lock word" ≠ "no shared data".

Shared data, read-only: **every CPU keeps the lines in S state** — only writes move cache lines.

rw_semaphore vs. RCU at a Glance

	rw_semaphore	RCU
Readers run together	yes	yes
Reader updates shared lock state	**usually yes**	**usually no**
Cache-line contention among readers	**can happen**	**very little**
Writer and readers at the same time	no — the writer excludes readers	**yes**
How the writer changes data	exclusive access	new version, then publish
Freeing old data	after unlock	after a grace period
What a reader sees	the protected current state	OLD or NEW

rw_semaphore, writer arrives: waits for every reader to leave; new readers wait for it.

RCU, writer arrives: publishes a new version — existing readers keep going, nobody is stopped.



rw_semaphore: readers in parallel. **RCU: readers in parallel — and no shared-lock contention**.

RCU in Three Ideas

The RCU idea: readers never wait for the writer, and the writer **delays freeing old data** until the readers using it are done.

1 Read

Readers read shared data with no lock contention: no waiting, no shared write.

2 Copy & Update

The writer builds a new copy, then swaps the shared pointer to it in one store.

3 Grace period

OLD is freed only after every reader that could still be using it has finished.

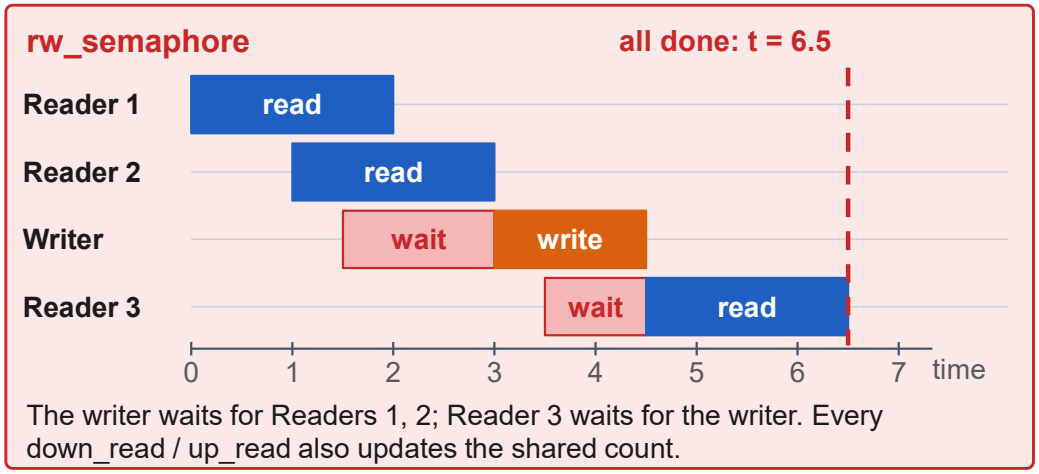
Coming up

Timeline	OLD and NEW	Four steps	Grace period	Preemptible RCU	Freeing OLD
rw_semaphore vs. RCU	the timeout example	read, copy, update, reclaim	quiescent states	context switch is not enough	synchronize_rcu, call_rcu

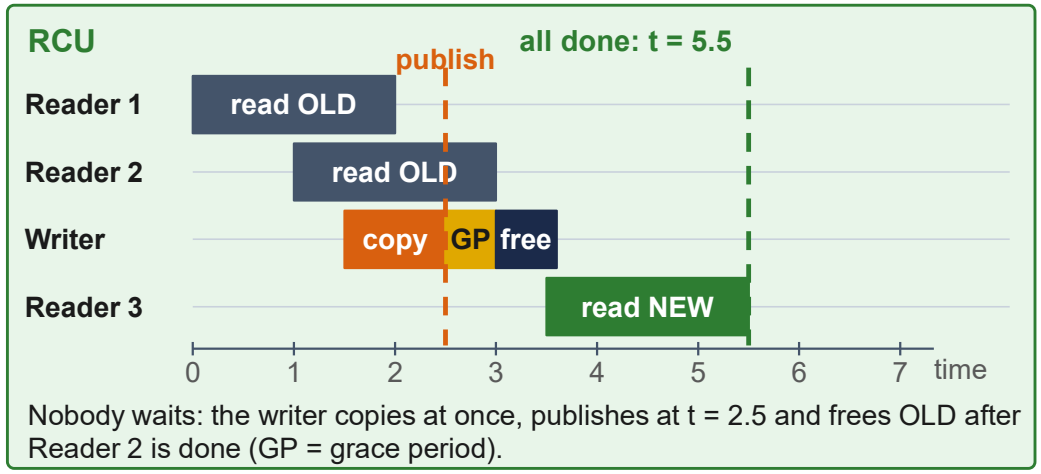
RCU: **readers go first; old data is freed later**.

Timeline: rw_semaphore vs. RCU

Same workload, two tools: with RCU, readers **never wait and never write a shared lock word** — and finish sooner.
Same arrivals in both (Example): Reader 1 at t = 0, Reader 2 at t = 1, writer at t = 1.5, Reader 3 at t = 3.5; a read takes 2 units.



The writer waits for Readers 1, 2; Reader 3 waits for the writer. Every down_read / up_read also updates the shared count.



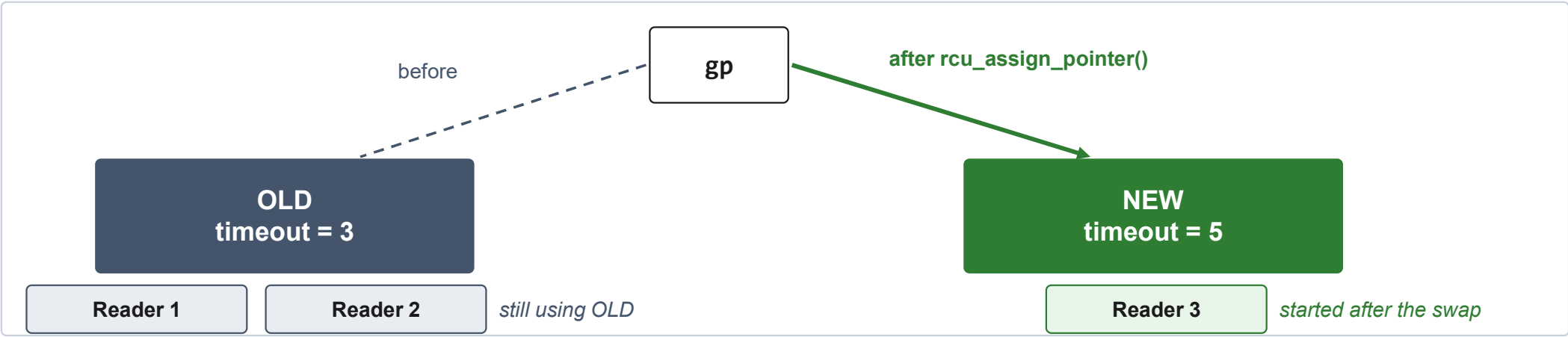
Nobody waits: the writer copies at once, publishes at t = 2.5 and frees OLD after Reader 2 is done (GP = grace period).

	rw_semaphore	RCU
Reader cost	atomic update of the shared count → line moves	**no atomic, no shared write**
Reader waits?	yes — while a writer holds it	**never**
Writer cost	take the lock	**copy + wait for the grace period**
What readers see	the protected current state	old or new copy, for a short time

RCU moves the cost **from readers to writers** — ideal when reads vastly outnumber updates.

The timeout Example: OLD and NEW Side by Side

Writer wants timeout = 3 → 5. Instead of editing the struct in place, it builds NEW and swaps the pointer.

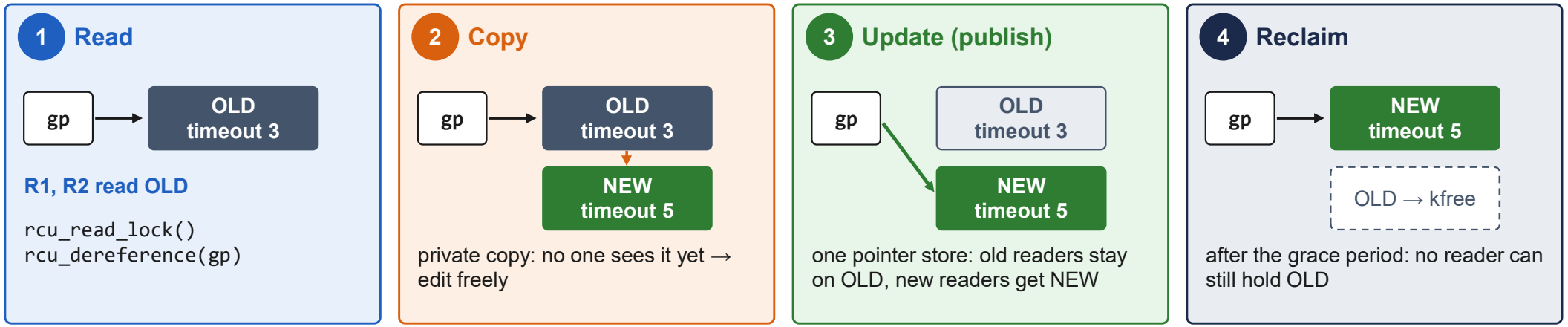


- Readers 1 and 2 may keep reading timeout = 3 (OLD)
- The writer publishes timeout = 5 (NEW)
- Reader 3 loads the new pointer and uses timeout = 5

Each reader sees OLD or NEW — never a half-updated struct (if it uses the one pointer it loaded). Readers pay almost nothing; the writer pays for the copy and for freeing OLD later.

Two versions can live at once — **who sees which depends on when they loaded the pointer**.

RCU Update in Four Steps: Read, Copy, Update, Reclaim



Reader

```
rcu_read_lock();          // begin read section
c = rcu_dereference(gp); // load the pointer
use(c->timeout);          // no sleeping here
rcu_read_unlock();        // end read section

// no atomic, no lock word, no waiting
```

Writer

```
new = kmalloc(sizeof(*new), GFP_KERNEL); // before lock
spin_lock(&cfg_lock);      // writers still exclude
old = rcu_dereference_protected(gp,
        lockdep_is_held(&cfg_lock));
*new = *old; new->timeout = 5; // 2 Copy + edit
rcu_assign_pointer(gp, new); // 3 Update: publish
spin_unlock(&cfg_lock);
synchronize_rcu();  // after unlock: may sleep
kfree(old);                // 4 Reclaim
```

Readers see **the whole OLD or the whole NEW copy** — never a half-updated one.

Grace Period in Non-preemptible RCU

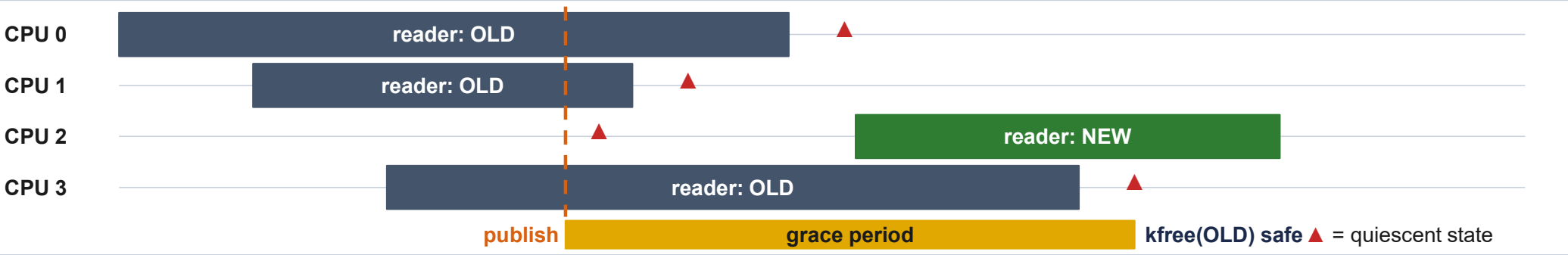
What rcu_read_lock() does (non-preemptible RCU)

- **Non-preemptible RCU:** it just disables preemption — no atomic, no shared write
- **So a reader cannot be switched out** until rcu_read_unlock()
- **Only marks "I am reading"** — it never blocks anyone
- **Rule:** do not sleep inside (need to sleep → SRCU)

Grace period

= the time until every reader that started before the publish has finished.

The kernel does not track readers one by one. In non-preemptible RCU a reader runs with preemption off, so once a CPU context-switches, enters user mode or goes idle — a **quiescent state** — it holds no old reader. All CPUs passed one → grace period over.



No need to wait for new readers: a reader that starts after the publish sees NEW, so the grace period ignores it (CPU 2).

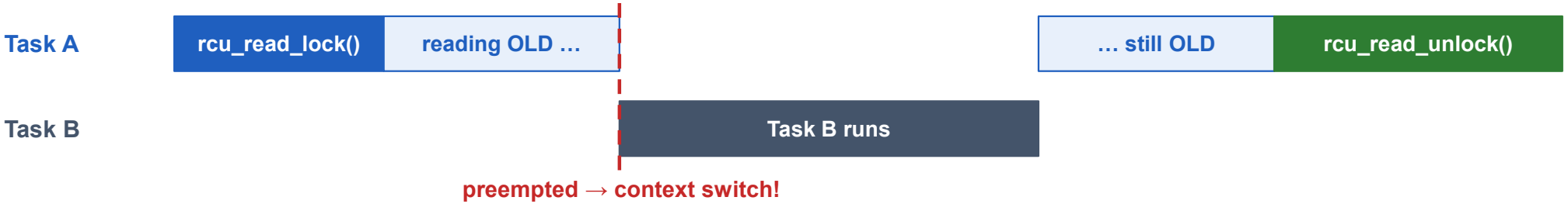
Quiescent states (non-preemptible RCU): the CPU ① does a context switch, ② enters user mode, or ③ goes idle. Preemptible RCU: next slide.

Readers pay almost nothing; **the writer pays by waiting** — OLD is freed only after every old reader is gone.

Preemptible RCU: A Context Switch Is Not Enough

Non-preemptible RCU: a reader cannot be switched out → a context switch proves it is done. **Preemptible RCU: it can.**

CPU 0 (preemptible RCU)



A context switch happened on CPU 0 — but Task A **may still hold OLD**. If the grace period ended here, freeing OLD would be a use-after-free.

So a context switch alone is not enough

A reader can be preempted in the middle of its read-side section and resume later on any CPU.

What Linux tracks instead

- Per task: how deep it is inside rcu_read_lock() (nesting)
- A list of readers preempted inside a read-side section
- Grace period ends only when CPUs are quiescent and those preempted readers have unlocked

Preemptible RCU (and PREEMPT_RT): **track the reader tasks, not just the CPUs**.

Same rcu_read_lock(), Two Implementations

The reader code is identical in both

```
rcu_read_lock();
p = rcu_dereference(gp);
use(p);
rcu_read_unlock();
```

Same API, different inside: which one you get is decided when the kernel is **built** (CONFIG_PREEMPT_RCU, next slide).

"Preemptible" = may be preempted, not "is always preempted". Most readers still run start to finish.

	Non-preemptible RCU	Preemptible RCU
Preemption inside a reader	not allowed	allowed
rcu_read_lock()	disables preemption	per-task RCU state
Can a reader be preempted?	no	yes
Context switch ⇒ reader done?	**yes**	**not by itself**
Grace period based on	CPU quiescent states	CPU states + preempted readers



One API, two engines — **the kernel build decides which one runs**.

Which RCU Does My Kernel Use? Check the Config

```
$ grep -E 'CONFIG_(PREEMPT_RCU|PREEMPT_NONE|PREEMPT_VOLUNTARY|PREEMPT|PREEMPT_RT)' /boot/config-$(uname -r)
```

Kernel config	RCU flavor
`CONFIG_PREEMPT_RCU=y`	**preemptible RCU**
`# CONFIG_PREEMPT_RCU is not set`	**usually non-preemptible RCU**
`CONFIG_PREEMPT_RT=y`	**preemptible RCU**

Example output

```
CONFIG_PREEMPT_RCU=y
CONFIG_PREEMPT_DYNAMIC=y
# CONFIG_PREEMPT_RT is not set
```

→ **built with preemptible RCU**

No /boot/config-*?
`zcat /proc/config.gz | grep CONFIG_PREEMPT_RCU`
(needs CONFIG_IKCONFIG_PROC)

PREEMPT_RT: RCU readers can be preempted too — a context switch never proves a reader is done; the kernel tracks each task.

Careful with CONFIG_PREEMPT_DYNAMIC: the scheduler's preemption mode can change at boot or run time (`cat /sys/kernel/debug/sched/preempt`). That is **not** the same as RCU preemptibility: a kernel built with CONFIG_PREEMPT_RCU=y keeps preemptible RCU even if the scheduler mode changes. Check **`CONFIG_PREEMPT_RCU`** first.

To know your RCU flavor, **look at CONFIG_PREEMPT_RCU** — not at the scheduler mode.

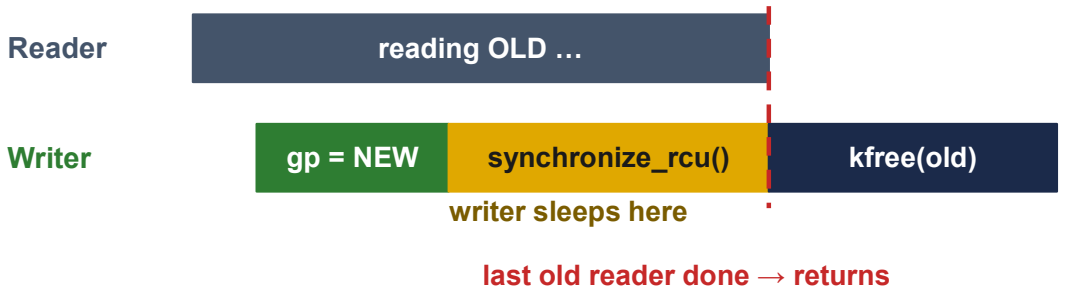
Freeing OLD (1): synchronize_rcu() — the Writer Waits

Writer code

```
rcu_assign_pointer(gp, new);  // 1 publish NEW
synchronize_rcu();    // 2-4 wait: grace period
kfree(old);           // 5 free OLD
```

1. Writer points gp at NEW
2. Writer calls synchronize_rcu()
3. Writer waits until every old reader is done
4. Grace period over → synchronize_rcu() returns
5. Writer frees OLD with kfree(old)

Synchronous reclaim



The writer itself waits for the grace period, then frees.

Key: the writer waits — and may sleep.

Never call synchronize_rcu() while holding a spinlock or in atomic context (IRQ handler, preemption off). Unlock first, then wait.

synchronize_rcu() = **synchronous reclaim**: simple, but the writer blocks.

Freeing OLD (2): call_rcu() and kfree_rcu() — No Waiting

call_rcu(): ask the kernel to free later

```
struct config {
    int timeout;
    struct rcu_head rcu;      // hook for RCU
};
void free_cb(struct rcu_head *head) {
    struct config *p =
        container_of(head, struct config, rcu);
    kfree(p);
}
rcu_assign_pointer(gp, new);
call_rcu(&old->rcu, free_cb);  // returns at once
```

"When every reader using OLD is done, please run free_cb()."

The writer goes on at once; the kernel's RCU callbacks free OLD after the grace period.

Note: callbacks usually run where sleeping is not allowed — cleanup that must sleep needs another mechanism.

kfree_rcu(): no callback to write

```
rcu_assign_pointer(gp, new);  kfree_rcu(old, rcu);
```

Function	Writer waits?	How OLD is freed
`synchronize_rcu()`	yes (may sleep)	writer calls kfree() after it returns
`call_rcu()`	no	callback runs after the grace period
`kfree_rcu()`	no	freed automatically after the grace period

Just freeing an RCU object? **kfree_rcu() is the simplest** — and the writer never waits.

PART 2: CPU Performance and Scheduling

- **Week 6**

- Why Do We Need Locks?
  - Shared data → the counter++ race → atomics and their cost → memory ordering
- Spinning and Sleeping Locks
  - spinlock, qspinlock → mutex, rwsem, semaphore → futex → granularity, contention
- Advanced Locking: RCU
  - Readers without locks → read, copy, update, reclaim → the grace period
- **Concurrency Bugs**
  - Atomicity, order, deadlock → hidden ABBA → checkpoint: find the bugs
- Tools for Concurrency Bugs
  - lockdep, ThreadSanitizer → debugging hangs → perf lock, c2c, trace

A Taxonomy of Concurrency Bugs

1 Data race

Unsynchronized access, at least one write

2 Atomicity violation

Check-then-act split across two critical sections

3 Order violation

Code assumes A happens before B, nothing enforces it

4 Deadlock

Threads wait for each other in a cycle

5 Lost wake-up

Signal arrives between check and sleep

6 Starvation / inversion

Someone never (or too late) gets the lock or CPU

What real bugs look like

~97%

of non-deadlock bugs were atomicity or order violations

Lu et al., "Learning from Mistakes", ASPLOS 2008 (MySQL, Apache, Mozilla, OpenOffice)

Most bugs involve only two threads and few variables — small tests can expose them.

Locking every access is not enough — the lock must cover the whole invariant, in the right order.

4 Bugs

Atomicity Violation: Check-Then-Act

Buggy: each access is "locked", the pair is not

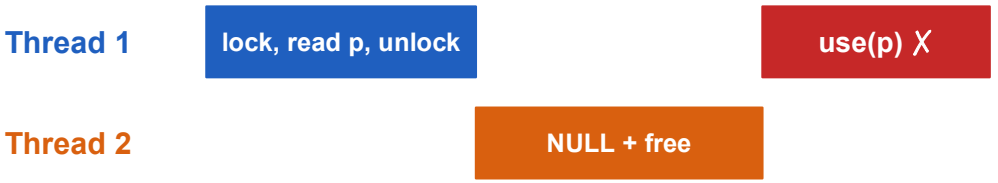
```
// Thread 1 (reader)
lock(&m);
    p = cache->entry;                          critical section
unlock(&m);
if (p != NULL)
    use(p->data);    // p may be freed!        ← outside the lock
// Thread 2 (evictor)
lock(&m);
    old = cache->entry; cache->entry = NULL;   critical section
unlock(&m);
free(old);
```

Fixed: one critical section, or hold a reference

```
lock(&m);
    p = cache->entry;
    if (p) ref_get(p);      // keep it alive   one critical section
unlock(&m);
if (p) { use(p->data); ref_put(p); }
```

lock / unlock inside the lock unsafe use

Interleaving



use-after-free: rare and timing-dependent

Often shows up only under load or on more cores.

Find the invariant

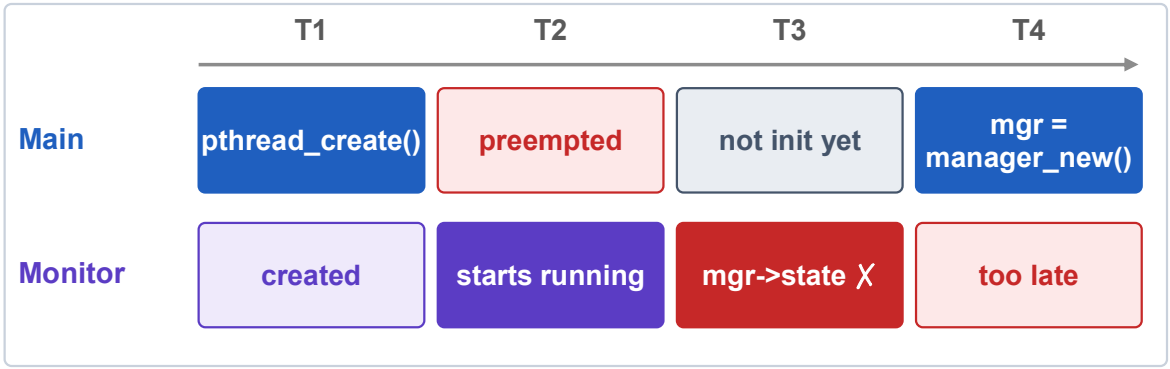
- Ask: which facts must stay true together? ("entry is valid while I use it")
- The lock (or a reference) must span the whole invariant, not each line
- Check and act inside the same critical section

Locks protect **invariants**, not individual variables.

Order Violation: Assuming "It Is Already Initialized"

✗ Buggy code

```
manager_t *mgr = NULL;            // global
// Main thread
pthread_create(&t, NULL, monitor, NULL);
mgr = manager_new();              // A: init
// Monitor thread
state = mgr->state;               // B: use
```



Expected: create thread → A: init → B: use

But pthread_create() does **not** guarantee the main thread keeps running first.

mgr is still NULL → **NULL pointer dereference (crash)**. Init and use are not synchronized → also a **data race**.

1 Why tests miss it

On an idle dev board the "expected" order wins 99.9% of the time — the bug looks fixed.

2 What exposes it

Load, a different core placement, a slower LITTLE core, or removing debug logging that used to delay the monitor.

Creating a thread does **not** mean the creator runs first — **the new thread may run at once**.

Fixing Order Violations: Init First, or pthread_once

✓ Fix 1: initialize first, then create the thread

```
// Main thread
mgr = manager_new();                    // init first
pthread_create(&t, NULL, monitor, mgr); // pass it
```

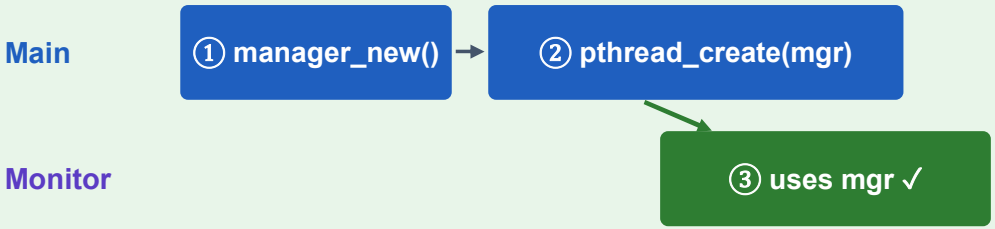
```
void *monitor(void *arg) {
    manager_t *mgr = arg;      // use the argument
    state = mgr->state;        // already initialized
    return NULL;
}
```

✓ Fix 2: many threads, lazy init → pthread_once()

```
static pthread_once_t once = PTHREAD_ONCE_INIT;
pthread_once(&once, init_manager); // any thread
void init_manager(void) { mgr = manager_new(); }
```

- init_manager() runs exactly once, even if many threads call it
- Callers wait until the init finishes
- Signature must be void init_manager(void); design failure handling separately

Order is now guaranteed



Even if the monitor runs first, it sees an initialized object.

Other ways to enforce order

- Condition variable / event: signal "ready"
- pthread_join(): wait until a thread finishes
- Release / acquire flag (p14)
- Create threads only after init (Fix 1)

If B must happen after A, **write code that makes it so** — do not rely on timing.

Deadlock: The ABBA Pattern

```
// Thread 1
lock(&A);
lock(&B);   // waits for T2
...
unlock(&B); unlock(&A);
```

```
// Thread 2
lock(&B);
lock(&A);   // waits for T1
...
unlock(&A); unlock(&B);
```

Wait-for graph



waits for B / waits for A

cycle → deadlock

Coffman condition	Break it by
Mutual exclusion	lock-free / RCU for read paths
Hold and wait	acquire all locks up front, or drop before taking the next
No preemption	trylock + back off + retry
Circular wait	**global lock order (A before B, always) ← most practical**

Hidden ABBA → next slide

- Calling a callback / LS2 handler while holding a lock
- Signal handler taking a lock the thread already holds
- Lock taken in both IRQ and process context without _irqsave

Document a lock order and never call unknown code while holding a lock.

Hidden ABBA: Deadlocks You Cannot See in the Code

1 Callback under a lock

```
// Thread 1
lock(&A);
callback();  // inside: lock(&B)
unlock(&A);
// Thread 2
lock(&B);
lock(&A);    // A→B vs B→A
```

`callback()` may take lock B. If another thread holds B and wants A → **ABBA**.

Rule: Never call unknown code (callbacks, LS2 handlers) while holding a lock.

2 Signal handler takes the lock

```
lock(&A);    // thread holds A
  ── signal arrives ──
  handler() {
    lock(&A);  // waits for
  }            // itself!
```

The handler runs on the **same thread**, which holds A → it waits for itself forever.

Rule: pthread_mutex_lock() is not async-signal-safe: never call it in a handler.

3 IRQ and process share a lock

```
// process context, CPU 0
spin_lock(&A);
  ── IRQ on CPU 0 ──
  irq_handler() {
    spin_lock(&A); // spins
  }
```

The process context can release A only after the IRQ handler ends — **deadlock on one CPU**.

Rule: Kernel: take such a lock with spin_lock_irqsave() (p18).

A deadlock can hide in code you call — **keep callbacks, signal handlers and IRQs away from your locks**.

CHECKPOINT: Find the Bugs

```
void *consumer(void *arg) {
  pthread_mutex_lock(&m);
  if (count == 0)                    // (1)
    pthread_cond_wait(&cv, &m);
  item_t it = q[--count];
  write(log_fd, &it, sizeof it);     // (2)
  if (it.bad) {
    return NULL;                     // (3)
  }
  pthread_mutex_unlock(&m);
  process(it);
  return NULL;
}
```

Context: producer–consumer

- `m` — mutex that protects the queue
- `cv` — condvar: "data has arrived"
- `count` — number of items in the queue
- `q` — the queue of jobs

Three bugs — find them

Look at:

(1) the wait condition

(2) the work done under the lock

(3) every exit path

Answer on the next slide.



Review every lock for: predicate loop, work under the lock, and every exit path.

Answer: Three Bugs and the Fixed Code

1 if → while

A woken consumer may find the queue empty again: another consumer took the item first (or a spurious wake-up) → **q[--count] underflows to −1**.

2 I/O under the lock

`write()` may block. A 100 ms disk write = 100 ms in which **no producer can add and no consumer can take**. Unlock first, then write.

3 return without unlock

If it.bad, the thread returns holding m → **the next lock() waits forever**. A POSIX mutex is not released when its owner exits (robust mutexes aside).

✓ Fixed

```
void *consumer(void *arg) {
  pthread_mutex_lock(&m);
  while (count == 0)          // (1) re-check
    pthread_cond_wait(&cv, &m);
  item_t it = q[--count];
  pthread_mutex_unlock(&m);   // queue done
  write(log_fd, &it, sizeof it); // (2) no lock
  if (it.bad)                 // (3) already
    return NULL;              //     unlocked
  process(it);
  return NULL;
}
```

Real bounded queue: also signal not_full after taking an item.

Loop on the predicate, **keep slow work outside the lock**, and unlock on every path.

PART 2: CPU Performance and Scheduling

- **Week 6**

- Why Do We Need Locks?
  - Shared data → the counter++ race → atomics and their cost → memory ordering
- Spinning and Sleeping Locks
  - spinlock, qspinlock → mutex, rwsem, semaphore → futex → granularity, contention
- Advanced Locking: RCU
  - Readers without locks → read, copy, update, reclaim → the grace period
- Concurrency Bugs
  - Atomicity, order, deadlock → hidden ABBA → checkpoint: find the bugs
- **Tools for Concurrency Bugs**
  - lockdep, ThreadSanitizer → debugging hangs → perf lock, c2c, trace

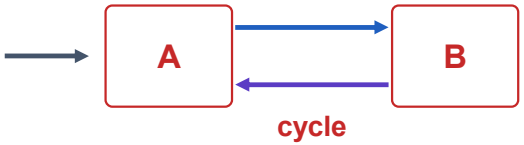
lockdep: Finding Deadlocks Before They Happen

lockdep (Lock Dependency Validator): tracks the order in which locks are taken at run time and detects potential **circular dependencies**.

Observed at run time (at different times)

T1 at 10:00: A → B

T2 at 10:05: B → A



→ **warning,** even though no deadlock actually happened

```
WARNING: possible circular locking
         dependency detected
kworker/1:2 is trying to acquire lock:
  (&dev->cfg_lock), at: dev_apply_cfg
but task is already holding lock:
  (&dev->io_lock), at: dev_io_work
 *** DEADLOCK ***  (example, abridged)
```

What it checks

- **Order per lock class:** all dev->io_lock instances are one class
- **Warns on the first inversion:** no need to hit the deadlock
- **IRQ-safety:** lock used in an IRQ but taken with IRQs on → warning

Limits and use

- **Sees only paths that actually ran** — it cannot find every possible deadlock
- **Test kernels:** CONFIG_PROVE_LOCKING, CONFIG_DEBUG_ATOMIC_SLEEP
- **Too slow for production images**

Turn a rare production hang into a deterministic warning in CI.

Data Race: What ThreadSanitizer Looks For

Data race: two or more threads access the same memory location concurrently, at least one writes, and nothing synchronizes them.
In C this is **undefined behavior** — and it appears only in some interleavings, so normal tests rarely catch it.

```
long counter = 0;
void *worker(void *arg) {
    counter++;   // T1 and T2
    return NULL; }
```

counter++ = ① read counter ② add 1 ③ store

Step	Thread 1	Thread 2
1	read counter (0)	
2		read counter (0)
3	store counter = 1	
4		store counter = 1

Two increments, result 1 → **lost update**

How TSan decides: the happens-before relation — order guaranteed by synchronization, not just by wall-clock time

```
pthread_mutex_lock(&m);
counter++;
pthread_mutex_unlock(&m);
```

✓ **Same mutex:** T1 unlock → T2 lock creates happens-before → **not a race**

✗ **No mutex:** nothing orders the two writes → **TSan reports a data race**, even if the run happened to print the right value

TSan does not warn because two threads use the same variable — it warns because no synchronization orders their accesses.

A race is **a missing happens-before** between conflicting accesses — not just a bad result.

ThreadSanitizer: Build, Run, Read the Report

```
$ gcc -g -O1 -fsanitize=thread -pthread race.c -o race
$ ./race
```

Option	Meaning
`-g`	debug info: source lines, function names
`-O1`	basic optimization
`-fsanitize=thread`	insert TSan instrumentation
`-pthread`	compile and link for POSIX threads

At run time TSan tracks every memory access and sync event; conflicting accesses with no happens-before → report.

```
WARNING: ThreadSanitizer: data race
  Write of size 8 by thread T2:
    #0 worker race.c:9
  Previous write of size 8 by thread T1:
    #0 worker race.c:9
  Location is global 'counter' of size 8
```

T2 wrote 8 bytes · **T1 wrote the same place before** · **`race.c:9`** = where · **`'counter'`** = the shared variable

Tool	Where	Finds	Cost / note
ThreadSanitizer	user space (GCC / Clang)	data races	~5–15× slower (instrumented)
Helgrind / DRD	user space (Valgrind)	data races, lock misuse	no rebuild, much slower
KCSAN	Linux kernel	kernel data races (sampling)	debug kernels
lockdep	Linux kernel	deadlock risk, lock order, IRQ rules	debug kernels (p60)

Use it well: run unit / stress tests under TSan in CI; fix, don't suppress ("benign" races break on Arm). **Limit:** finds races only on paths that actually ran.

Races are timing-dependent; sanitizers make them deterministic enough to fix.

Debugging Hangs (1): User Stacks and Kernel Wait Points

1 GDB: every thread's user-space call stack

```
$ gdb -p 2210 -batch -ex "thread apply all bt"
```

`-p 2210` attach to PID 2210 · **`-batch`** run, then exit · **`thread apply all bt`** backtrace of all threads

```
Thread 3: __futex_abstimed_wait ... lock_db()
Thread 1: __futex_abstimed_wait ... lock_cache()
Thread 2: read() ...  (held: cache_mutex)
```

- Thread 3 waits for the DB lock; Thread 1 waits for the cache lock
- **Thread 2 holds cache_mutex while doing read()** — slow I/O under a lock delays everyone
- **Note:** a backtrace does not always show who owns a lock; "held" comes from extra analysis

2 /proc/…/wchan: where in the kernel it sleeps

```
$ for t in /proc/2210/task/*; do
    echo "$t $(cat "$t/wchan")"; done
```

```
/proc/2210/task/2210 futex_wait_queue
/proc/2210/task/2213 folio_wait_bit_common
```

wchan	Likely meaning
`futex_wait_queue`	waiting on a lock (futex)
`folio_wait_bit_common`	waiting for a page / file I/O

- Each thread: /proc/PID/task/TID/; wchan = the kernel function it sleeps in
- Separates "sleeping on a lock" from "sleeping on I/O"
- A function name alone does not prove the cause

GDB shows **where in your code** each thread waits; wchan shows **where in the kernel**.

Debugging Hangs (2): Hung Task Detector and SysRq

3 Hung task detector: tasks stuck too long

```
INFO: task svc_worker:2213 blocked for more
      than 120 seconds.
Call Trace:
  ... mutex_lock+0x2c ...
```

Set the threshold (seconds):

```
# echo 120 > /proc/sys/kernel/hung_task_timeout_secs
```

- Watches mainly **D-state** tasks (TASK_UNINTERRUPTIBLE)
- A normal futex wait is interruptible → may **not** be reported
- **A warning ≠ deadlock:** slow I/O can trigger it too

4 SysRq w: dump every blocked task

```
# echo w > /proc/sysrq-trigger
# dmesg | tail
```

```
task:svc_worker state:D pid:2213
 __schedule
 schedule
 io_schedule ...
```

- `w` = Show Blocked Tasks: kernel stacks go to the kernel log
- `state:D` = uninterruptible sleep
- `io_schedule` → look at disk I/O, file system, driver or kernel locks
- **Whole-system view; needs root**

The kernel can **name the stuck task and its stack** — but a long wait is not always a deadlock.

Debugging a Hang: Which Tool, in Which Order

Tool	Shows	Answers
GDB thread apply all bt	user-space call stacks	Which thread waits in which function?
/proc/…/wchan	kernel wait function	Where in the kernel does it sleep — lock or I/O?
Hung task detector	tasks blocked for a long time	Which task has made no progress?
SysRq w	kernel stacks of blocked tasks	Which tasks are blocked, system-wide?

Example: a multithreaded server suddenly stops answering

1 All stacks (GDB)

Who waits in pthread_mutex_lock, pthread_cond_wait, read …?

→

2 wchan

Waiting on a futex — or on I/O?

→

3 Owner + dependencies

Not only who waits: what is the lock holder doing?

→

4 Kernel side

Hung task + SysRq: D-state tasks, kernel stacks

Find who waits, then **find what the holder is doing** — the cause is usually there.

Observing Contention (1): perf lock and perf c2c

1 perf lock contention: kernel lock waits

```
$ sudo perf lock contention -a -b -- sleep 5
```

```
 contended   total wait   max wait   type   caller
      1523       842 ms      31 ms  mutex  svc_update+0x4c
       310        24 ms       2 ms  spin   queue_rq+0x20
```

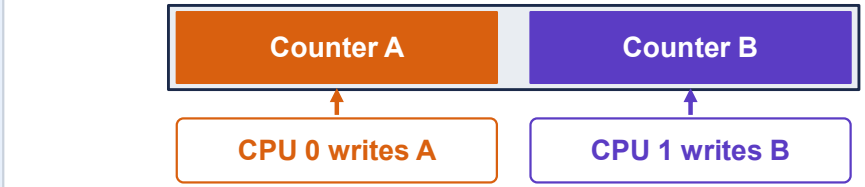
Field	Meaning
`contended`	how many times the lock was contended
`total wait`	total time spent waiting
`max wait`	longest single wait
`type / caller`	lock kind / where it happened

`svc_update()`: 1,523 contentions, 842 ms waiting → the bottleneck, in numbers.

Note: mainly kernel locks — not a direct view of user-space pthread_mutex contention.

2 perf c2c: cache-line sharing (false sharing)

```
$ sudo perf c2c record -a -- sleep 5
$ sudo perf c2c report --stdio
```



No data race (different variables) — but each write invalidates the other CPU's copy → **the line ping-pongs** (p9).

c2c clue: many hits on one line at different offsets.

Fix: padding / alignas(64), or per-CPU counters.

Needs hardware memory-access sampling — not on every CPU.

Measure before you fix: **which lock, how long** — and **which cache line**.

Observing Contention (2): perf trace, and Which Tool When

3 perf trace: where user-space threads wait

```
$ sudo perf trace -s -p 2210 -- sleep 5
```

```
 syscall      calls   total(ms)
 futex         2341      812.5
 epoll_pwait    980      120.1
```

futex: 2,341 calls, 812.5 ms → a lot of time in futex system calls.

But ≠ lock contention: a condvar wait for work is also a futex sleep. Check the op (FUTEX_WAIT / WAKE), the call site and the thread stacks.

4 lockdep: lock-order risk

```
// T1: lock(A); lock(B);
// T2: lock(B); lock(A);
→ A→B and B→A observed
→ "possible circular locking"
```

Finds a dangerous order **before** the deadlock happens — but only on paths it has seen (details: p60).

Tool	Analyzes	Question it answers
`perf lock contention`	kernel lock waits	Which lock, and how long do we wait?
`perf c2c`	cache-line sharing	Is there false sharing?
`lockdep`	lock acquisition order	Is there a deadlock risk?
`perf trace`	system calls, wait time	Where do user-space threads wait?

Long futex time is a **symptom**, not a diagnosis — confirm with the op, the call site and the stacks.

Summary: Sharing Data Safely

1 Part 1 · Why locks

- counter++ = load, add, store
- Atomics: one variable, cache-line cost
- Arm reorders: release / acquire

2 Part 2 · Spinning and sleeping

- Can it sleep? spinlock vs. mutex
- futex: no syscall unless contended
- Small locks: shrink the serial part

3 Part 3 · RCU

- Readers take no lock
- Copy, publish, reclaim later
- Writers pay the grace period

4 Part 4 · Concurrency bugs

- Atomicity and order violations
- ABBA deadlock, also hidden ones
- Fix: one global lock order

5 Part 5 · Tools

- lockdep and TSan find bugs early
- Hang? Who waits for what?
- perf lock / c2c / trace: measure

Goal: **shared data that stays correct** — and threads that rarely wait for each other.

End of Week 6

Youngjae Kim, Ph.D.

